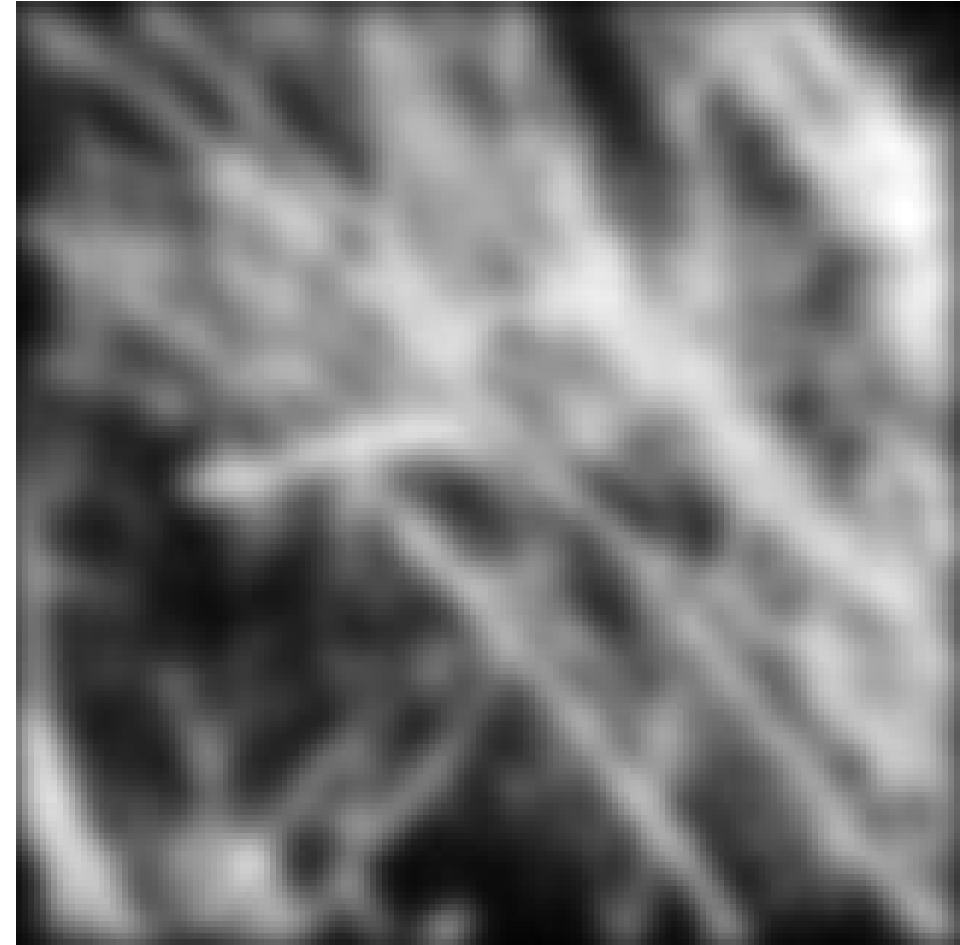

CMPE 297-01 Computer Vision

Image Filtering

Fall 2026, DMH 348

Image filtering



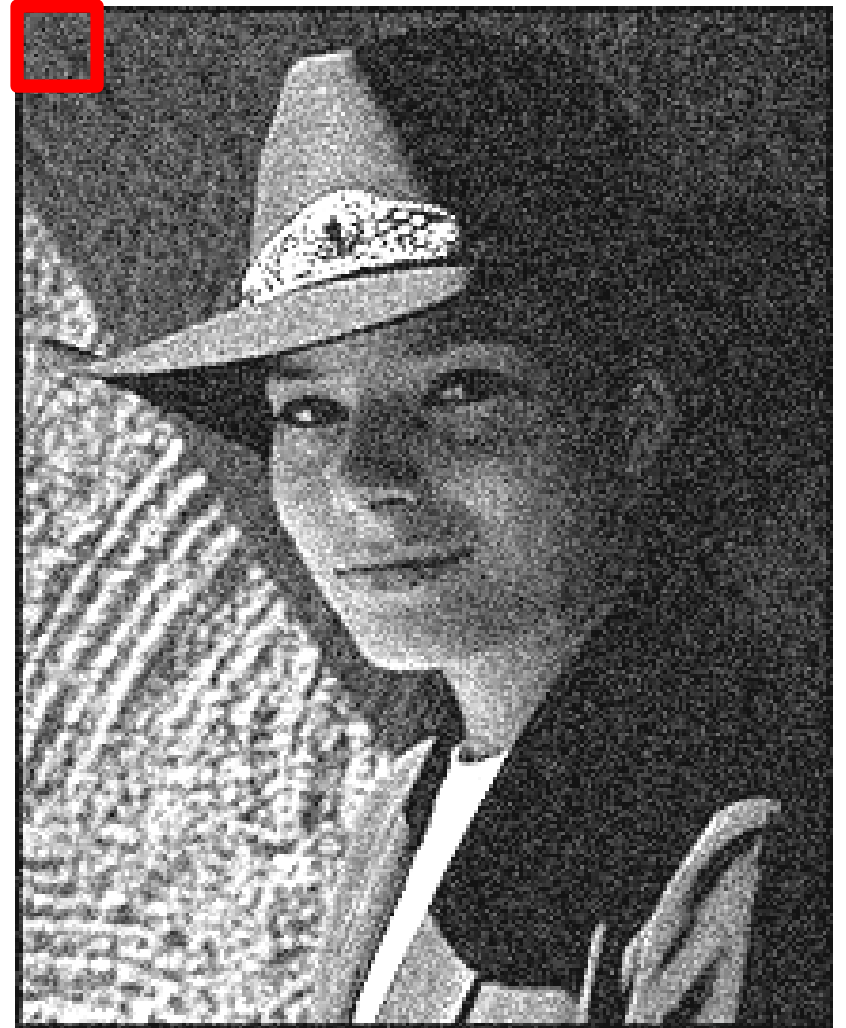
Lecture Outline

- Linear filtering and its properties
- Gaussian filters and their properties
- Nonlinear filtering: Median filtering, bilateral filtering
- Filtering application: Hybrid images

Linear Filtering and Its Properties

Sliding window operations

- Let's slide a fixed-size window over the image and perform the same simple computation at each window location
- Example use case: how do we reduce image noise?
 - Let's take the *average* of pixel values in each window
 - More generally, we can take a *weighted sum* where the weights are given by a *filter kernel*



Applying a linear filter

Input

I_{11}	I_{12}	I_{13}	I_{14}	I_{15}	I_{16}
I_{21}	I_{22}	I_{23}	I_{24}	I_{25}	I_{26}
I_{31}	I_{32}	I_{33}	I_{34}	I_{35}	I_{36}
I_{41}	I_{42}	I_{43}	I_{44}	I_{45}	I_{46}
I_{51}	I_{52}	I_{53}	I_{54}	I_{55}	I_{56}

Filter

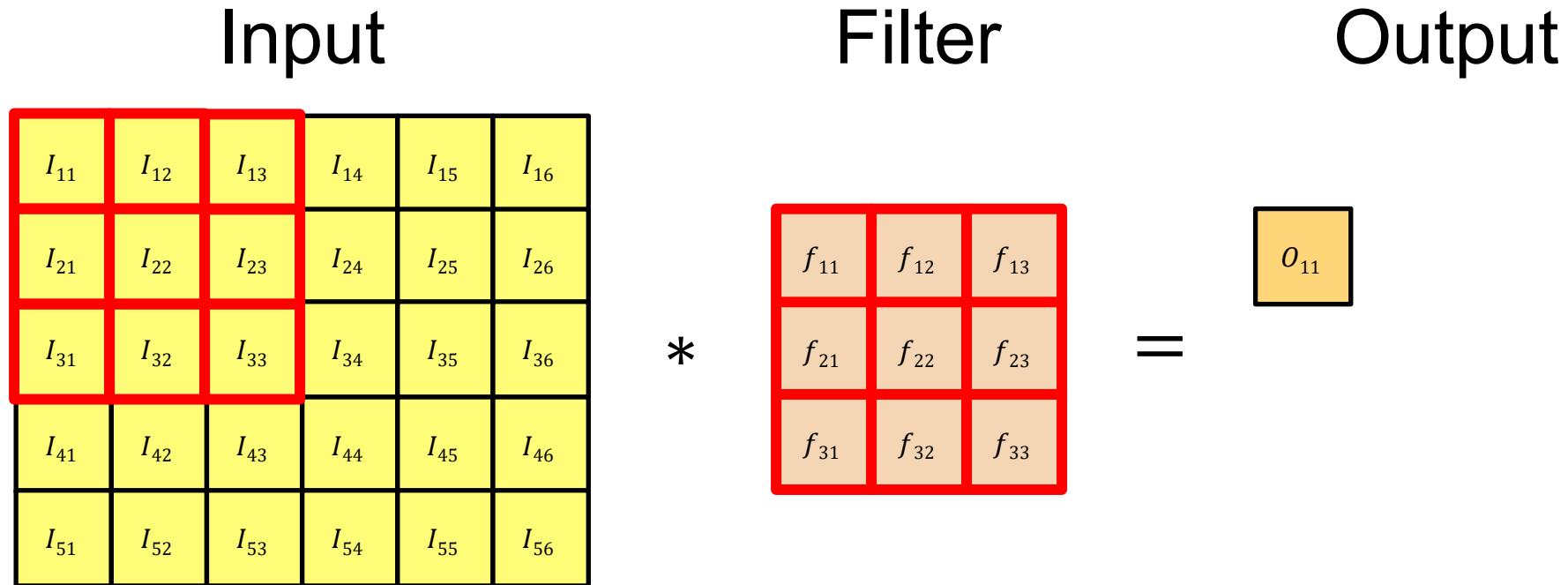
f_{11}	f_{12}	f_{13}
f_{21}	f_{22}	f_{23}
f_{31}	f_{32}	f_{33}

*

=

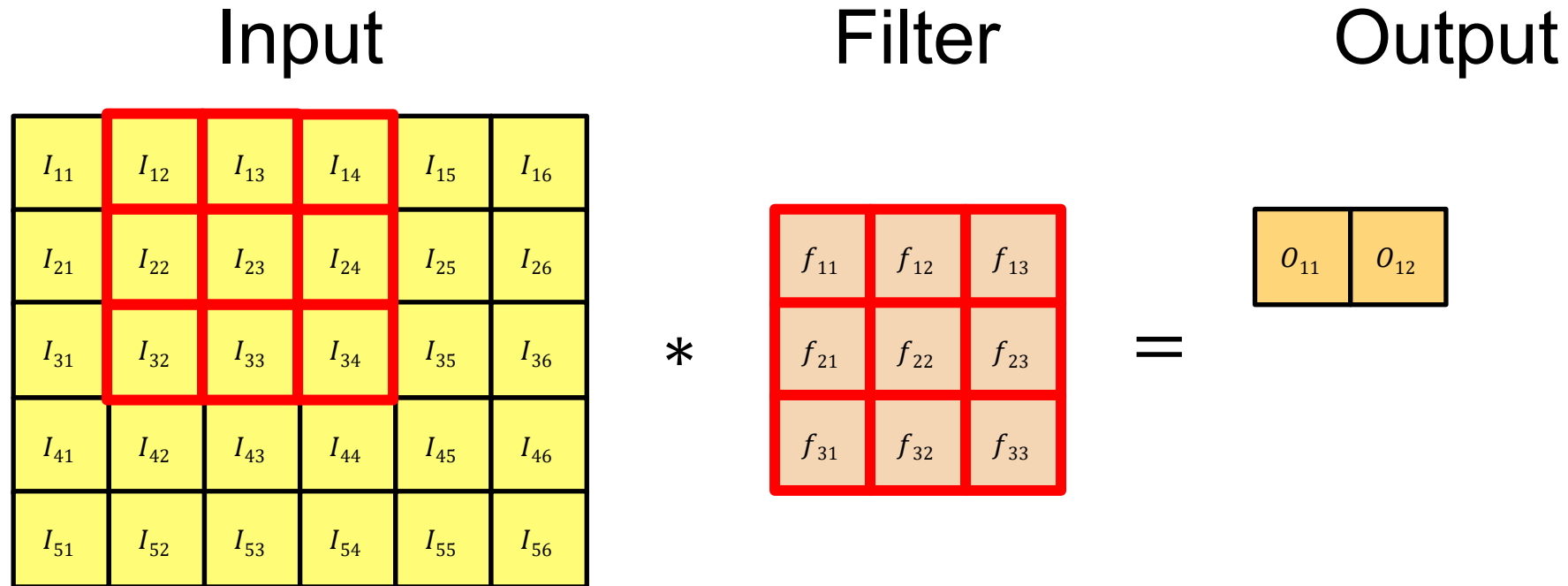
Output

Applying a linear filter



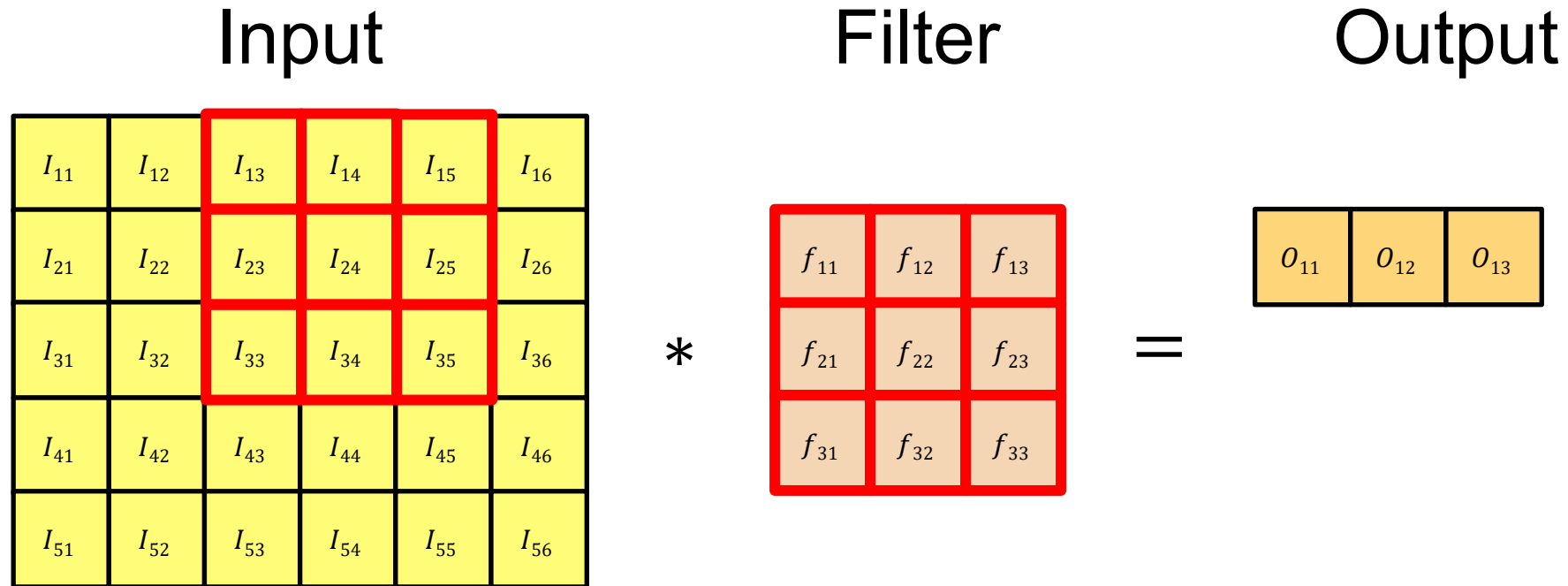
$$O_{11} = I_{11} \cdot f_{11} + I_{12} \cdot f_{12} + I_{13} \cdot f_{13} + \dots + I_{33} \cdot f_{33}$$

Applying a linear filter



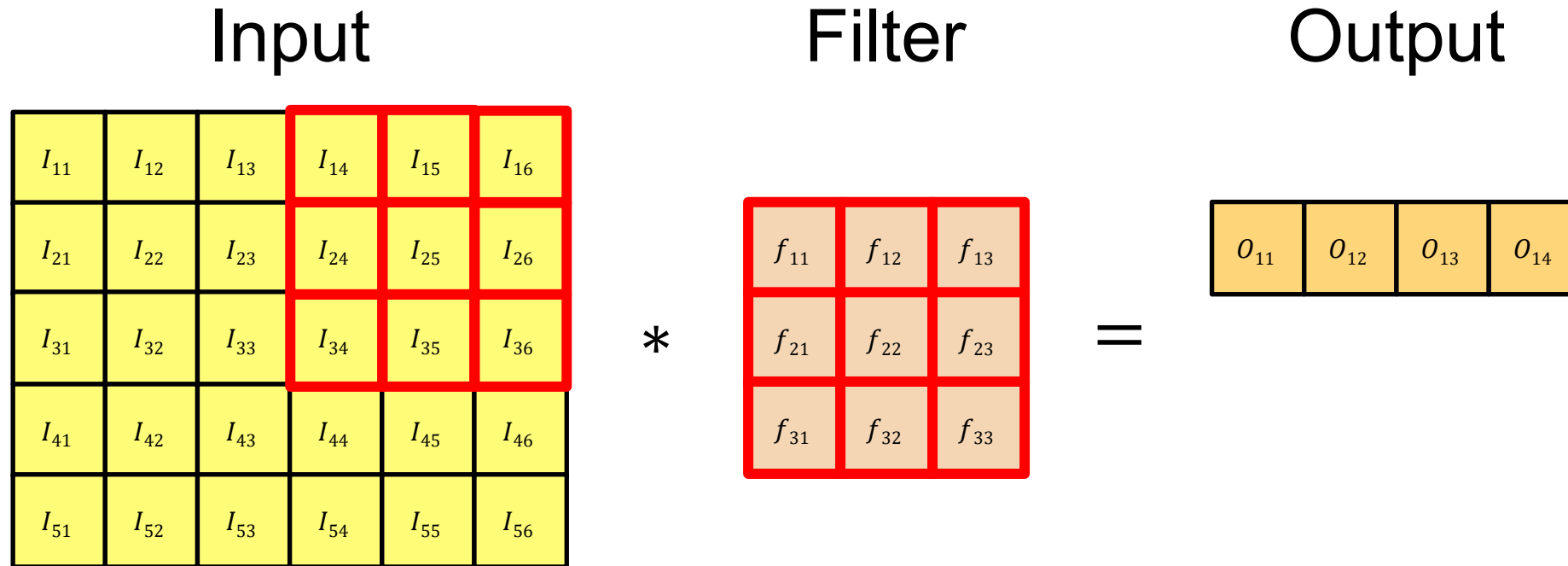
$$O_{12} = I_{12} \cdot f_{11} + I_{13} \cdot f_{12} + I_{14} \cdot f_{13} + \dots + I_{34} \cdot f_{33}$$

Applying a linear filter



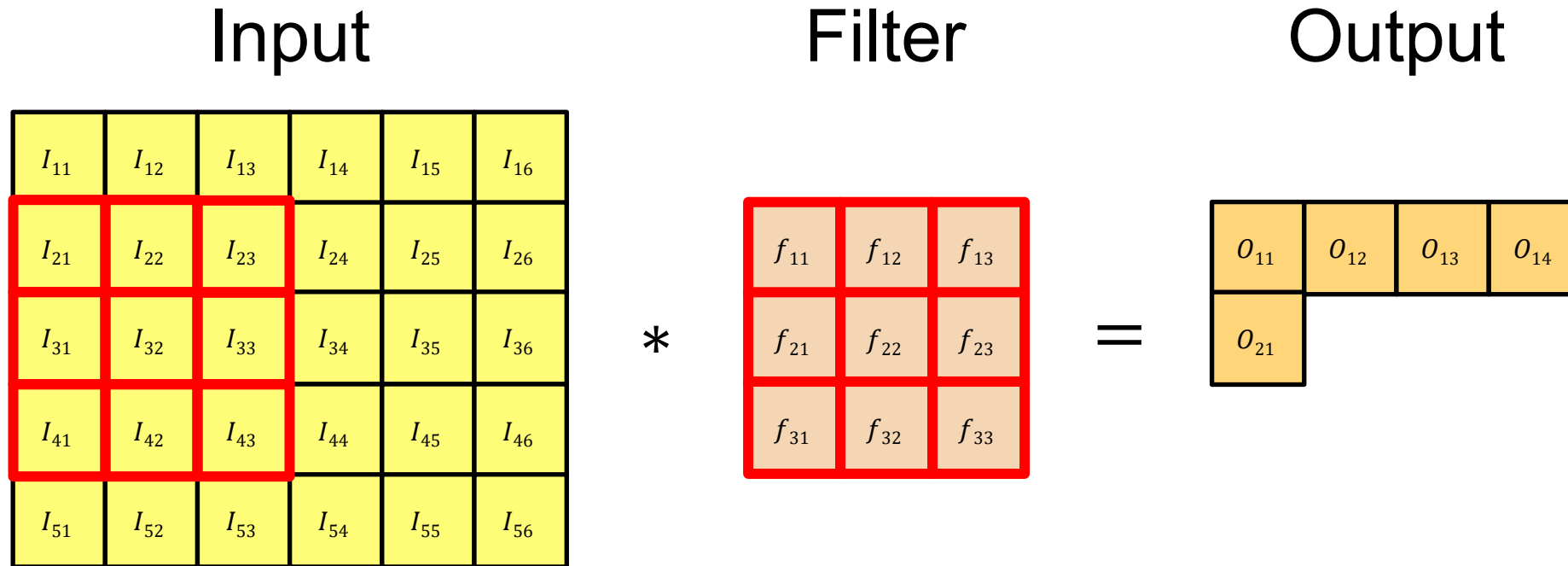
$$O_{13} = I_{13} \cdot f_{11} + I_{14} \cdot f_{12} + I_{15} \cdot f_{13} + \dots + I_{35} \cdot f_{33}$$

Applying a linear filter



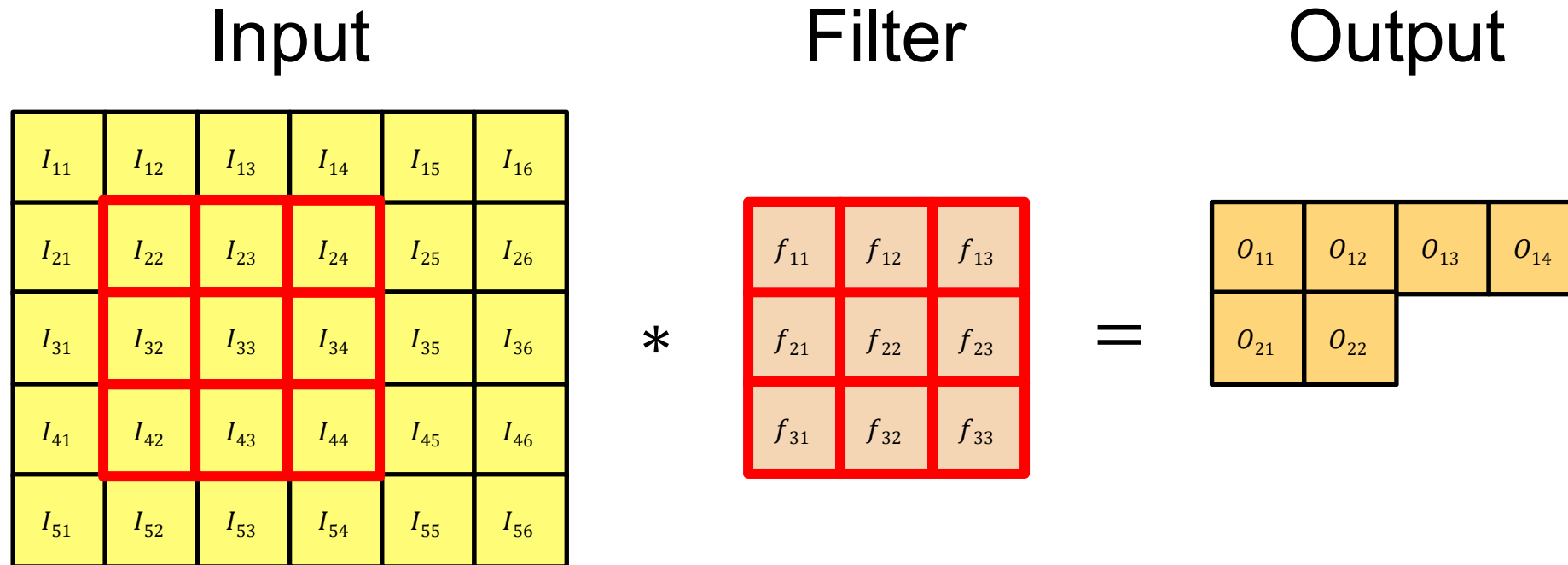
$$O_{14} = I_{14} \cdot f_{11} + I_{15} \cdot f_{12} + I_{16} \cdot f_{13} + \dots + I_{36} \cdot f_{33}$$

Applying a linear filter



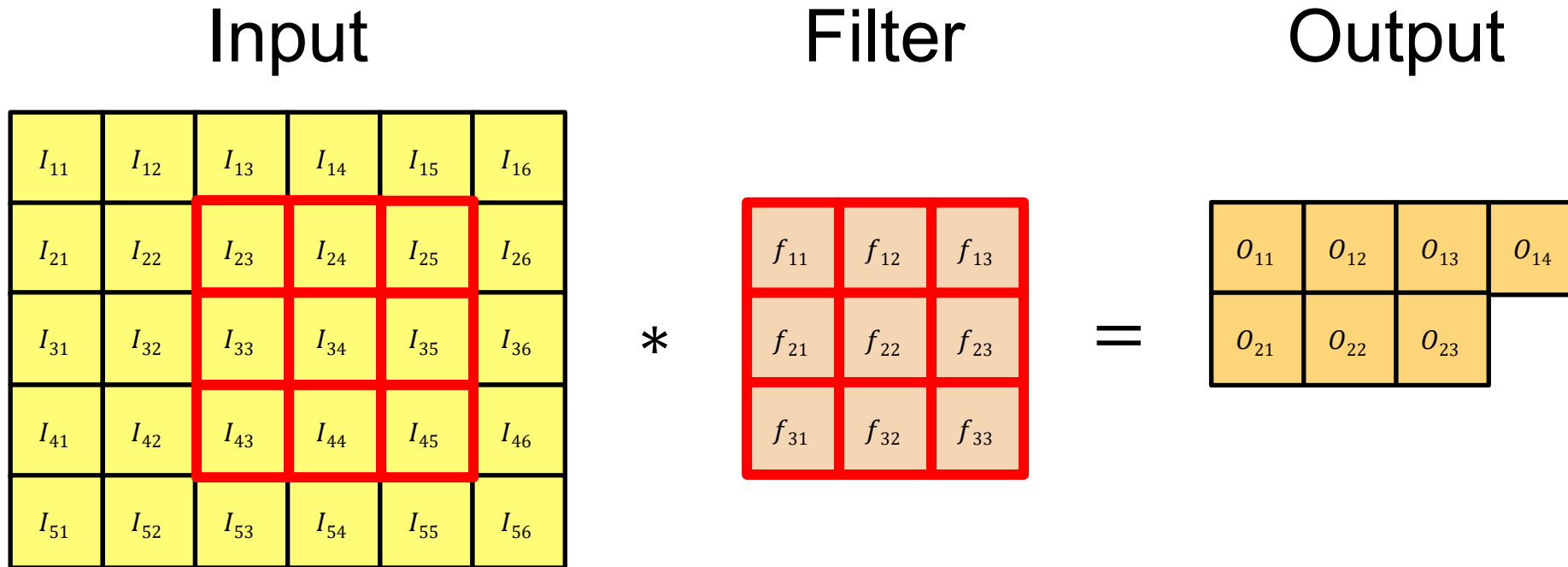
$$O_{21} = I_{21} \cdot f_{11} + I_{22} \cdot f_{12} + I_{23} \cdot f_{13} + \dots + I_{43} \cdot f_{33}$$

Applying a linear filter



$$O_{22} = I_{22} \cdot f_{11} + I_{23} \cdot f_{12} + I_{24} \cdot f_{13} + \dots + I_{44} \cdot f_{33}$$

Applying a linear filter



$$O_{23} = I_{23} \cdot f_{11} + I_{24} \cdot f_{12} + I_{25} \cdot f_{13} + \dots + I_{45} \cdot f_{33}$$

Applying a linear filter

Input

I_{11}	I_{12}	I_{13}	I_{14}	I_{15}	I_{16}
I_{21}	I_{22}	I_{23}	I_{24}	I_{25}	I_{26}
I_{31}	I_{32}	I_{33}	I_{34}	I_{35}	I_{36}
I_{41}	I_{42}	I_{43}	I_{44}	I_{45}	I_{46}
I_{51}	I_{52}	I_{53}	I_{54}	I_{55}	I_{56}

Filter

f_{11}	f_{12}	f_{13}
f_{21}	f_{22}	f_{23}
f_{31}	f_{32}	f_{33}

*

=

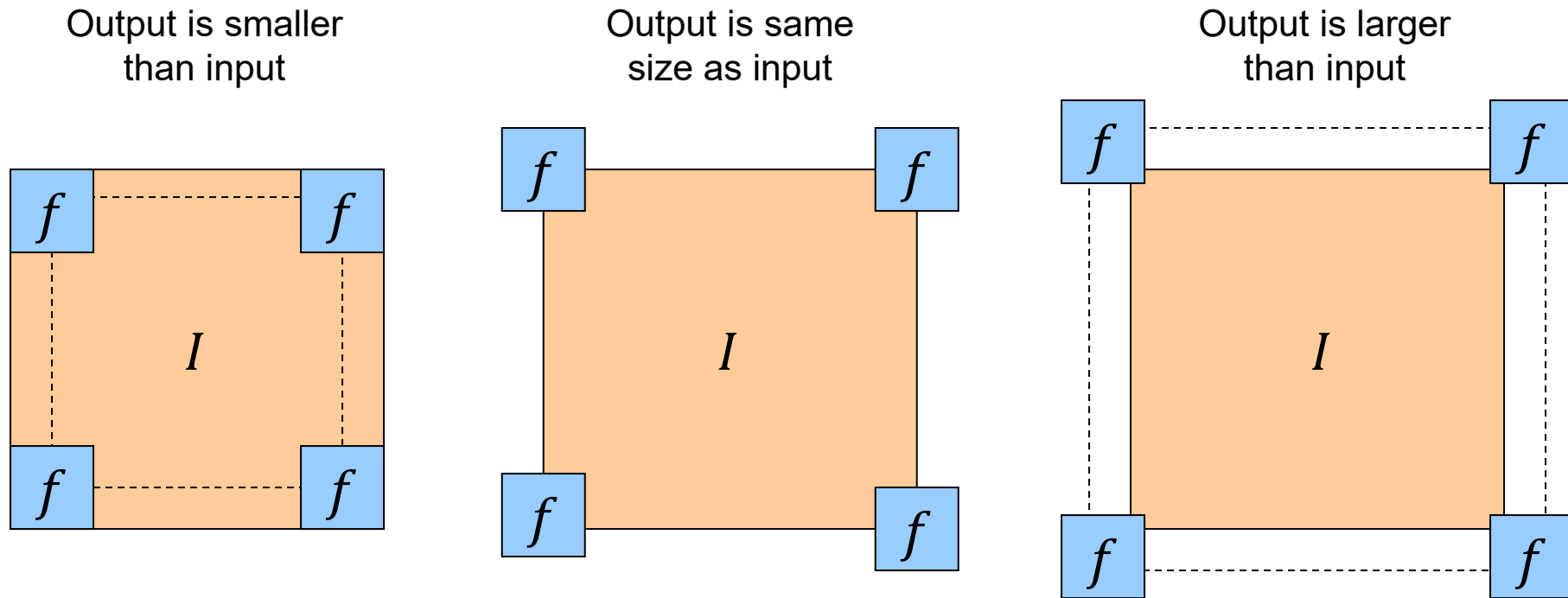
Output

O_{11}	O_{12}	O_{13}	O_{14}
O_{21}	O_{22}	O_{23}	O_{24}
O_{31}	O_{32}	O_{33}	O_{34}

What filter values should we use to find the average in a 3×3 window?

Practical details: Dealing with edges

- To control the size of the output, we need to use *padding*

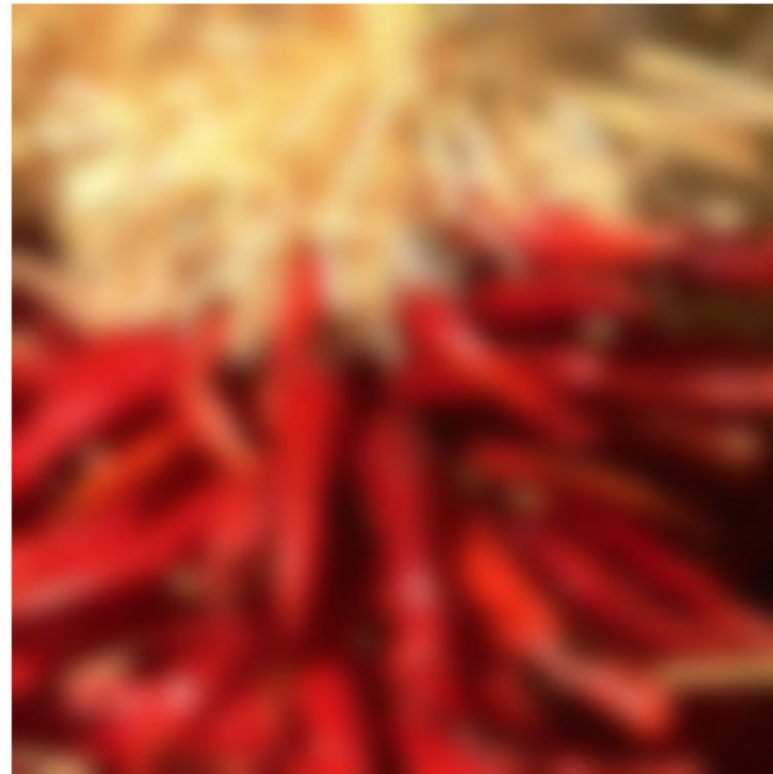


Practical details: Dealing with edges

- To control the size of the output, we need to use *padding*
- What values should we pad the image with?

Practical details: Dealing with edges

- To control the size of the output, we need to use *padding*
- What values should we pad the image with?
 - Zero pad (or clip filter)
 - Wrap around
 - Copy edge
 - Reflect across edge



Properties: Linearity

$$\text{filter}(I, f_1 + f_2) = \text{filter}(I, f_1) + \text{filter}(I, f_2)$$

$$\text{filter}\left(\begin{array}{|c|} \hline \blacksquare \\ \hline \end{array}, \begin{array}{|c|} \hline \blacksquare \\ \hline \end{array} + \begin{array}{|c|} \hline \square \\ \hline \end{array}\right) = \text{filter}\left(\begin{array}{|c|} \hline \blacksquare \\ \hline \end{array}, \begin{array}{|c|} \hline \square \\ \hline \end{array}\right) = \begin{array}{|c|} \hline \text{blurred square} \\ \hline \end{array}$$

$$\text{filter}\left(\begin{array}{|c|} \hline \blacksquare \\ \hline \end{array}, \begin{array}{|c|} \hline \blacksquare \\ \hline \end{array}\right) + \text{filter}\left(\begin{array}{|c|} \hline \blacksquare \\ \hline \end{array}, \begin{array}{|c|} \hline \square \\ \hline \end{array}\right) = \begin{array}{|c|} \hline \text{blurred square} \\ \hline \end{array} + \begin{array}{|c|} \hline \text{blurred square} \\ \hline \end{array} = \begin{array}{|c|} \hline \text{blurred square} \\ \hline \end{array}$$

Properties: Linearity

$$\text{filter}(I, f_1 + f_2) = \text{filter}(I, f_1) + \text{filter}(I, f_2)$$

Also:

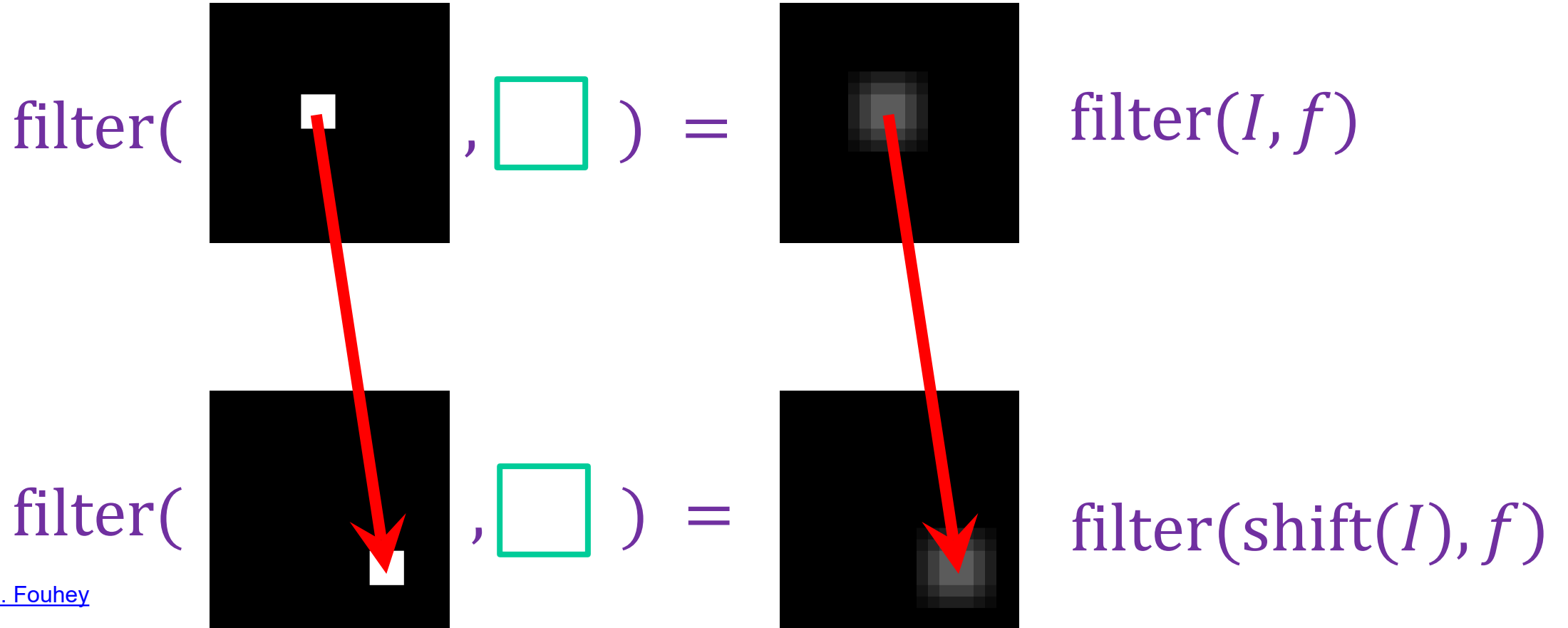
$$\text{filter}(I_1 + I_2, f) = \text{filter}(I_1, f) + \text{filter}(I_2, f)$$

$$\text{filter}(kI, f) = k \text{ filter}(I, f)$$

$$\text{filter}(I, kf) = k \text{ filter}(I, f)$$

Properties: Shift-invariance

$$\text{filter}(\text{shift}(I), f) = \text{shift}(\text{filter}(I, f))$$



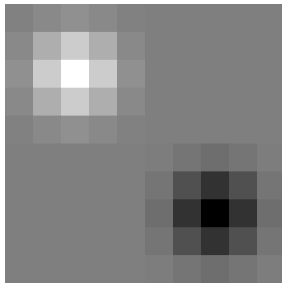
More linear filtering properties

- Commutativity: $f * g = g * f$
 - For infinite signals, no difference between filter and signal
- Associativity: $f * (g * h) = (f * g) * h$
 - Convolution of several filters one after another is equivalent to convolving with one combined filter:
$$\left((g * f_1) * f_2 \right) * f_3 = g * (f_1 * f_2 * f_3)$$
- Identity: for *unit impulse* e , $f * e = f$

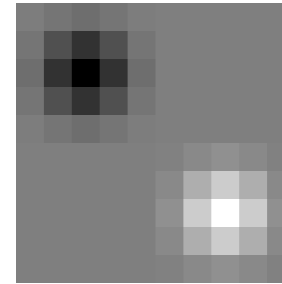
Note: Filtering vs. “convolution”

- In classical signal processing terminology, convolution is filtering with a *flipped* kernel, and filtering with an upright kernel is known as *cross-correlation*
 - Check convention of filtering function you plan to use!

Filtering or “cross-correlation”
(Kernel in original orientation)



“Convolution”
(Kernel flipped in x and y)



Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

One surrounded
by zeros is the
identity filter



Filtered
(no change)

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted *left*
By one pixel

Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

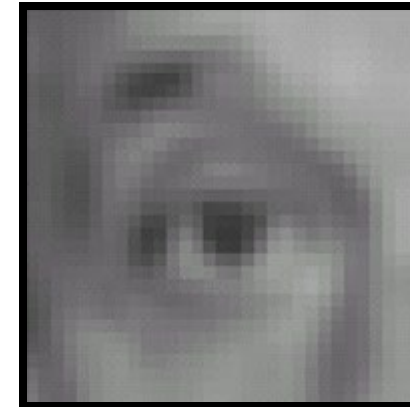
Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1



Blur (with a
box filter)

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

— $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

Practice with linear filters



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sharpening filter:

Accentuates differences
with local average

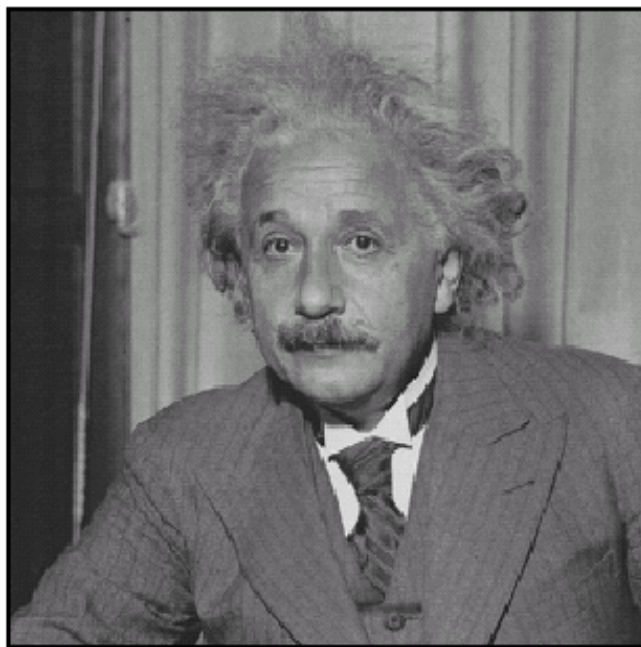
(Note that filter sums to 1)



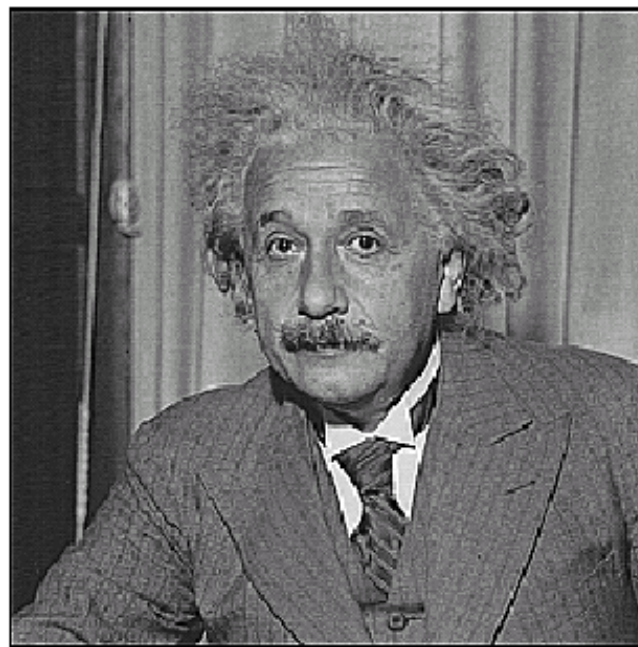
Sharpened

Sharpening

Original



Sharpened



Sharpening in more detail

Original

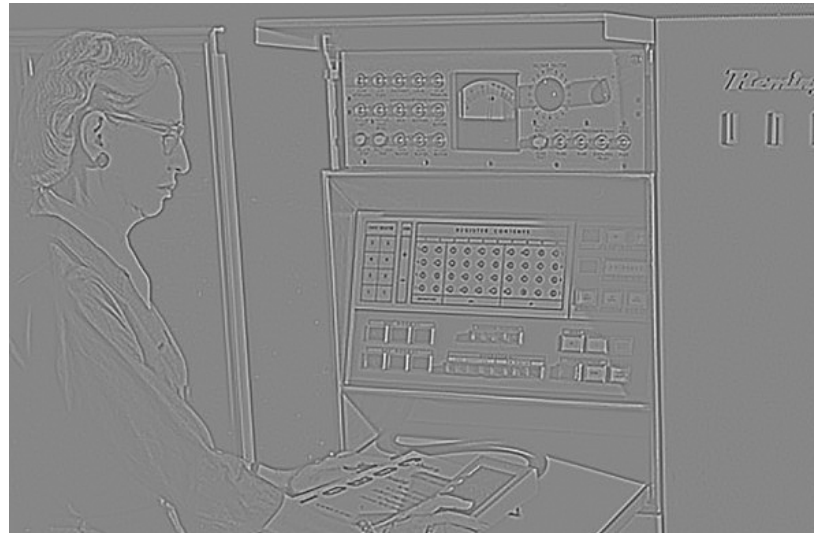


Smoothed



Details

=



Sharpening in more detail

Original



Details



$+ \alpha$

Sharpened ($\alpha = 1$)

=



Sharpening in more detail

Original



Details



$+ \alpha$

Sharpened ($\alpha = 2$)

=



Sharpening in more detail

Original



Details



$+ \alpha$

Sharpened ($\alpha = 10$)

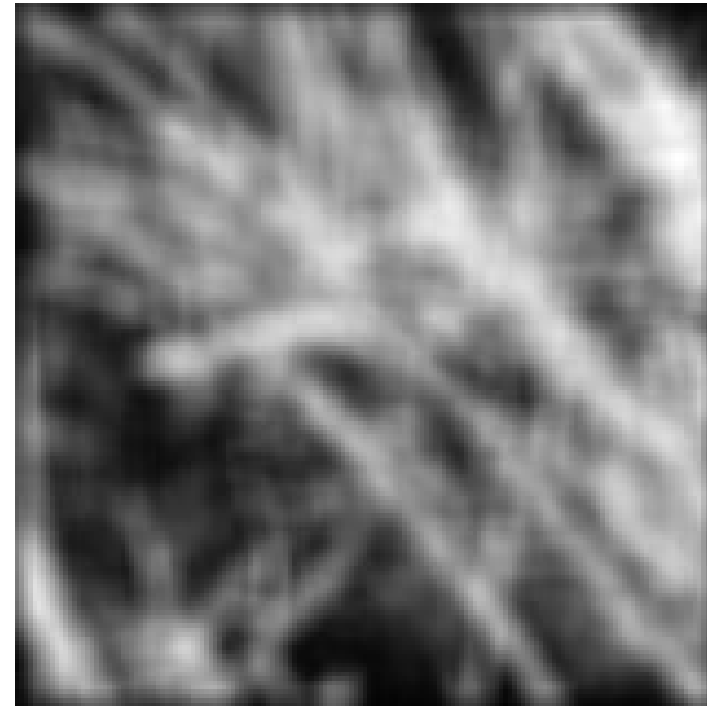
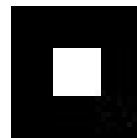
=



Gaussian Filters and Their Properties

Smoothing revisited

- What's wrong with this picture?
- What's the solution?



Smoothing revisited

- What's wrong with this picture?
- What's the solution?
 - To eliminate box artifacts, weight contribution of neighborhood pixels according to their closeness to the center

“proportional to”
(renormalize values to sum to 1)

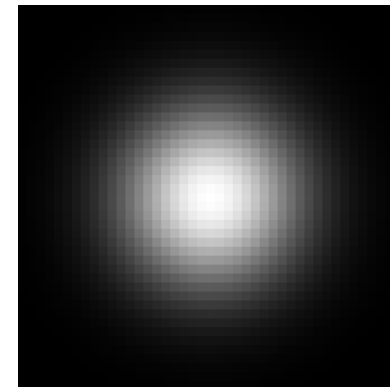
↓

$$G(x, y) \propto \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

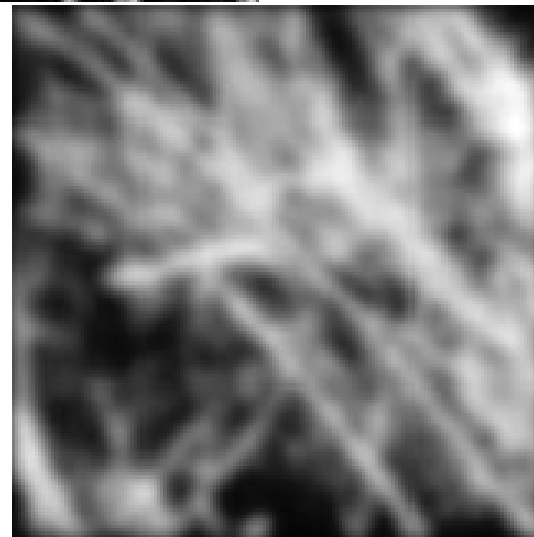
↑

standard deviation
(determines size of “blob”)

Gaussian filter

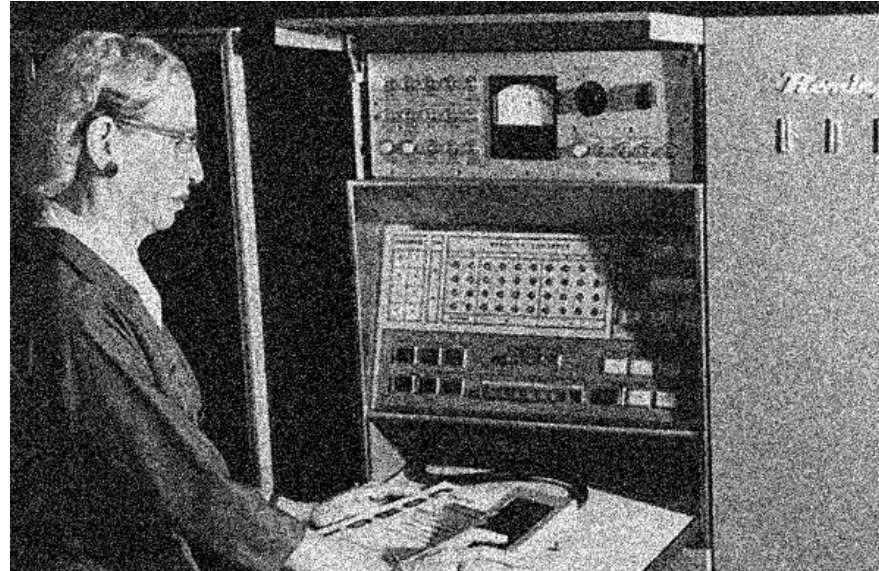


Gaussian vs. box filtering



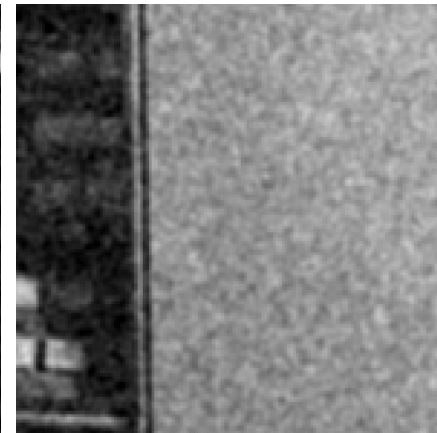
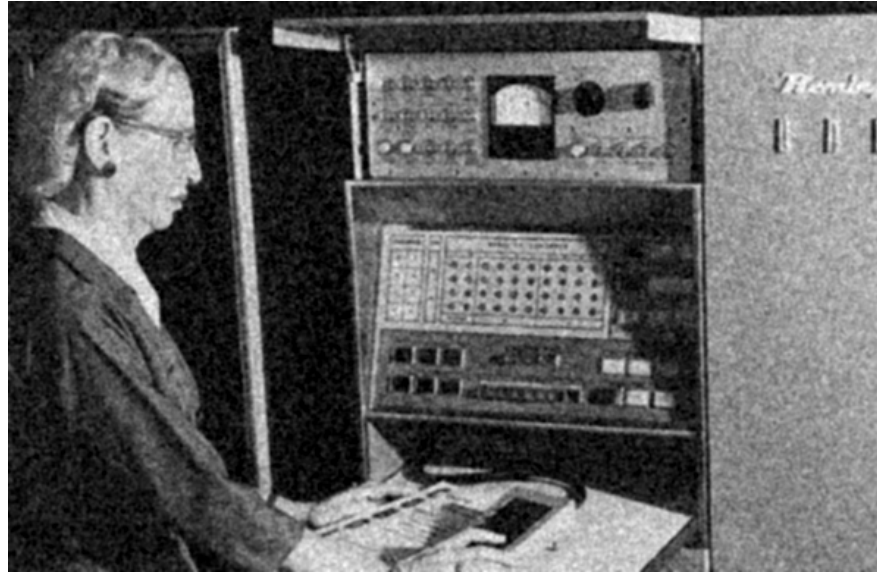
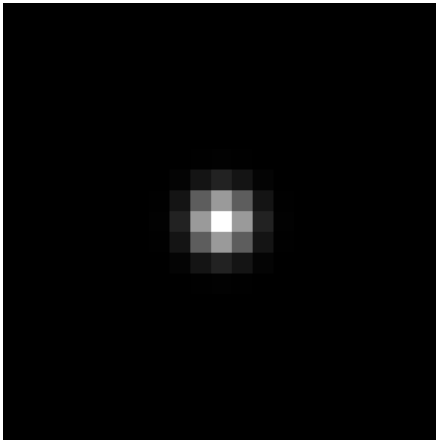
Applying Gaussian filters

Input image
(no filter)



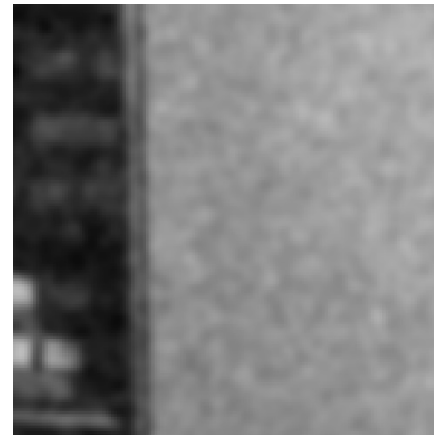
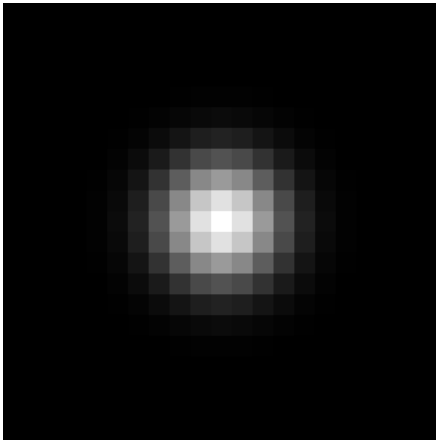
Applying Gaussian filters

$$\sigma = 1$$



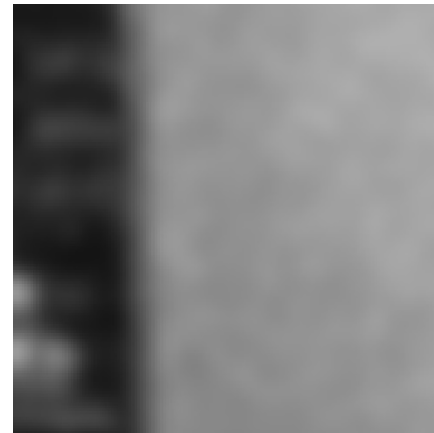
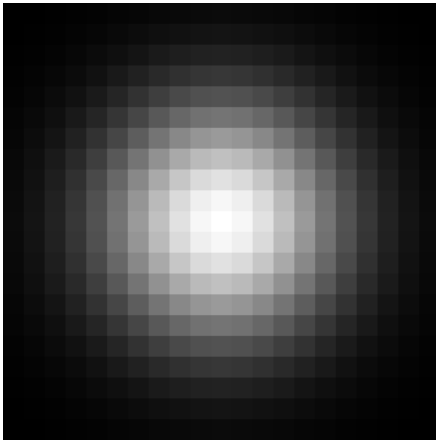
Applying Gaussian filters

$$\sigma = 2$$



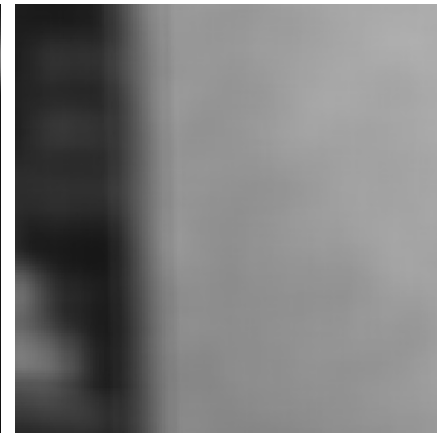
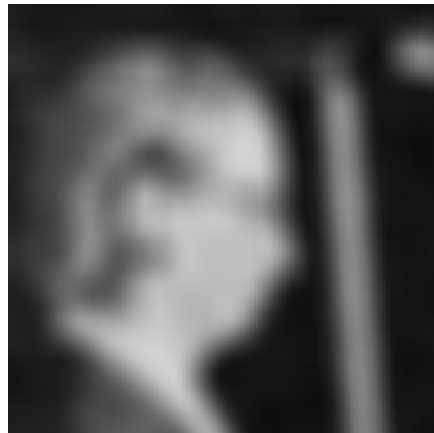
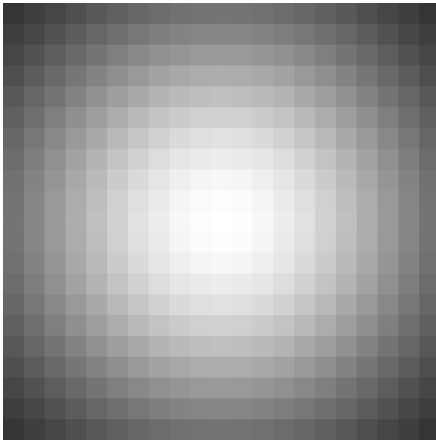
Applying Gaussian filters

$$\sigma = 4$$



Applying Gaussian filters

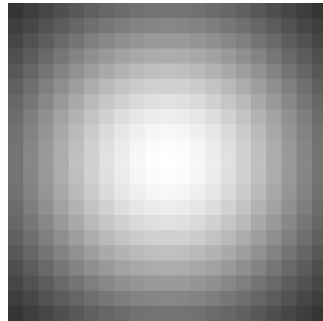
$$\sigma = 8$$



Choosing filter size

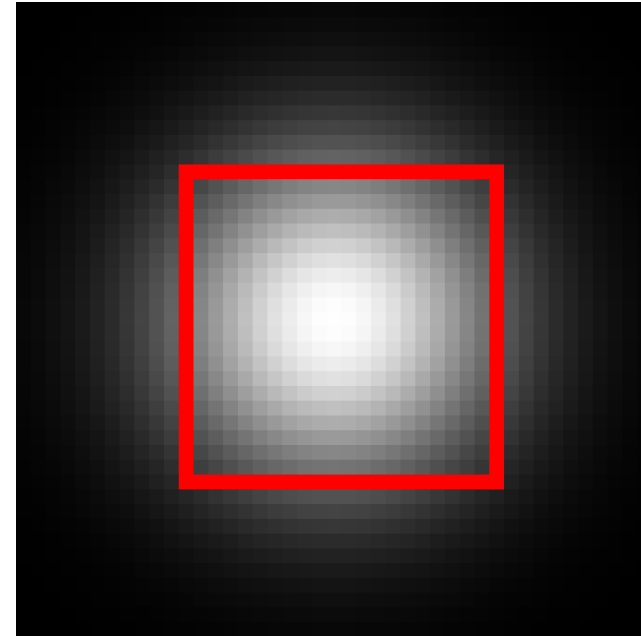
- Rule of thumb: set filter width to about 6σ (captures 99.7% of the energy)

$\sigma = 8$
Width = 21



Too small!

$\sigma = 8$
Width = 43



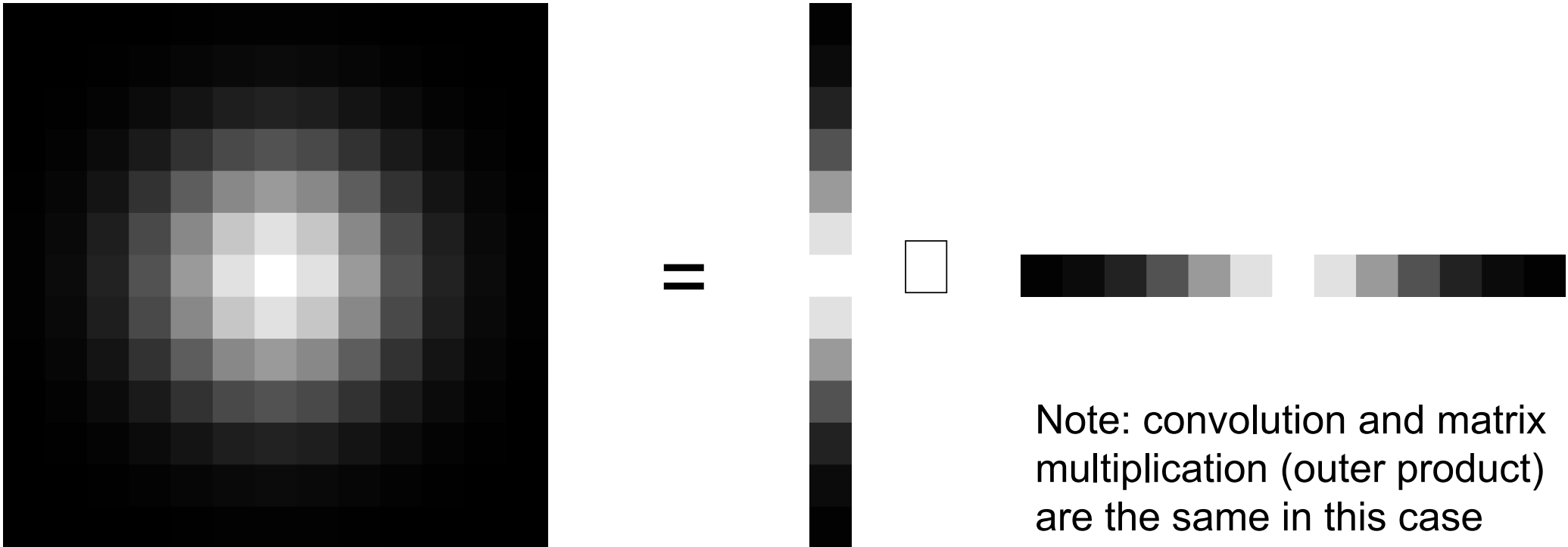
A bit small (might be OK)

Gaussian filters: Properties

- Gaussian is a *low-pass filter*: it removes high-frequency components from the image (more on this soon)
- Convolution with self is another Gaussian
 - So we can smooth with small- σ kernel, repeat, and get same result as larger- σ kernel would have
 - Convolving *two times* with Gaussian kernel with std. dev. σ is the same as convolving *once* with kernel with std. dev. $\sigma\sqrt{2}$
- Gaussian kernel is *separable*: it factors into product of two 1D Gaussians

Separability of the Gaussian filter

$$\frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right) \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{y^2}{2\sigma^2}\right)$$



Note: convolution and matrix multiplication (outer product) are the same in this case

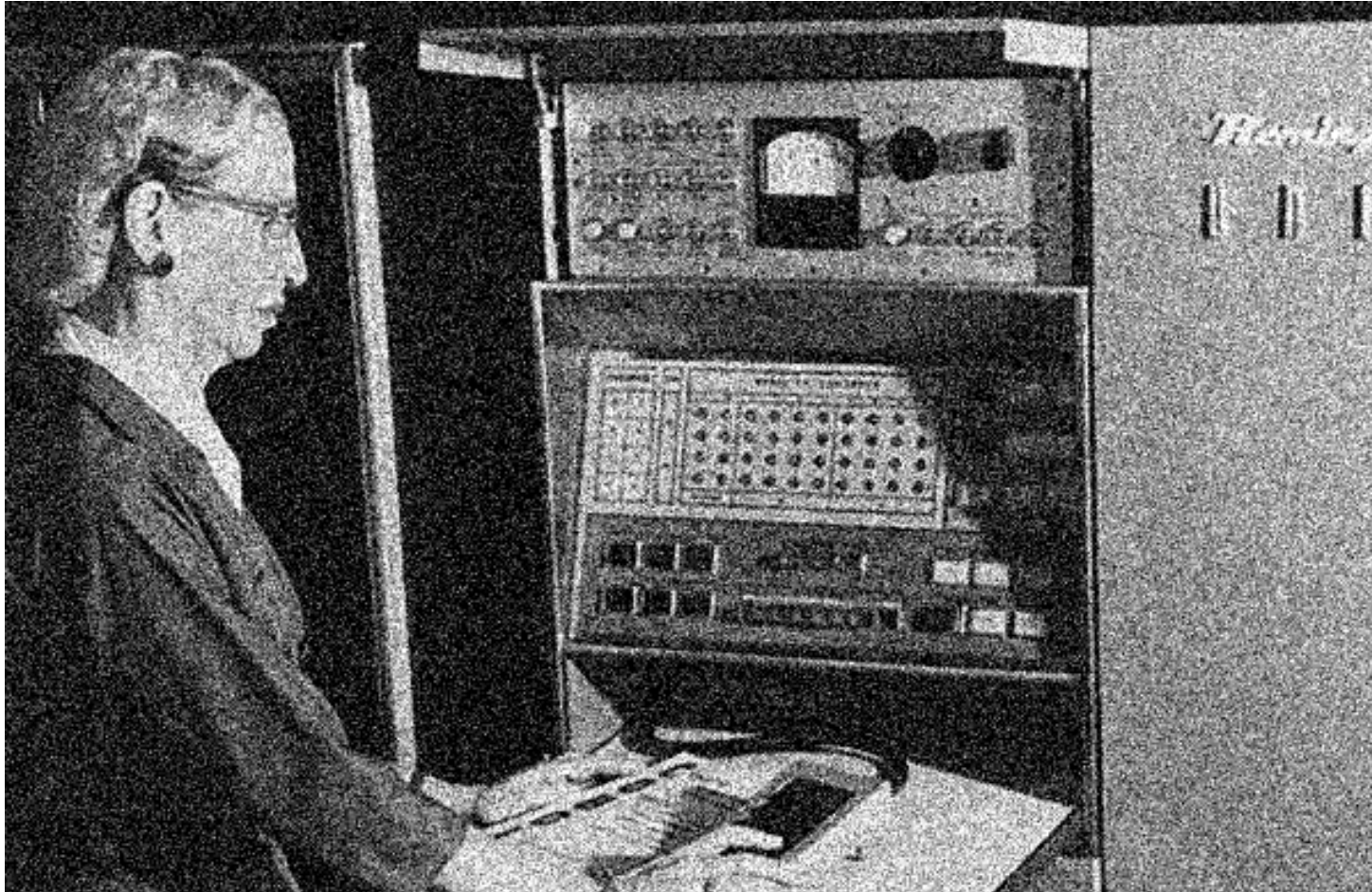
Why is separability useful?

- Separability means that a 2D convolution can be reduced to two 1D convolutions (one along rows and one along columns)
- What is the complexity of filtering an $n \times n$ image with an $m \times m$ kernel?
 - $O(n^2 m^2)$
- What if the kernel is separable?
 - $O(n^2 m)$

Nonlinear Filtering: Median Filtering

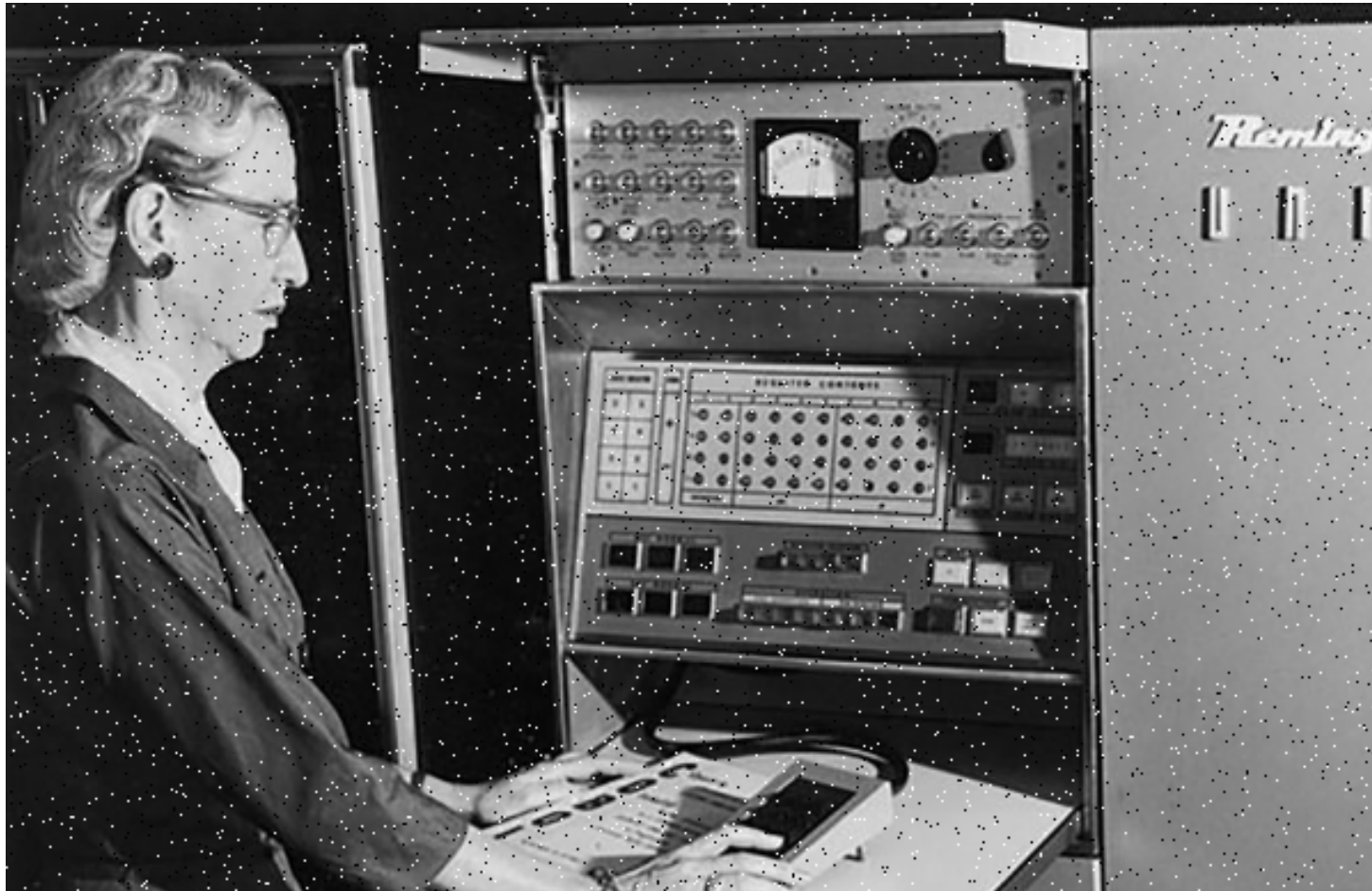
Different types of noise

- What kind of noise is Gaussian filtering appropriate for?
 - *Additive, zero-mean* noise (assuming nearby pixels share the same value)



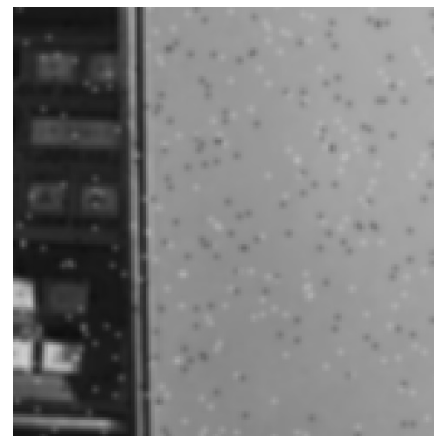
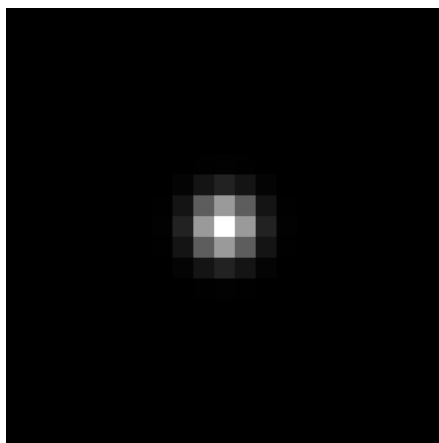
Different types of noise

- What about *impulse* or *shot noise*, i.e., when some pixels are arbitrarily replaced by spurious values?



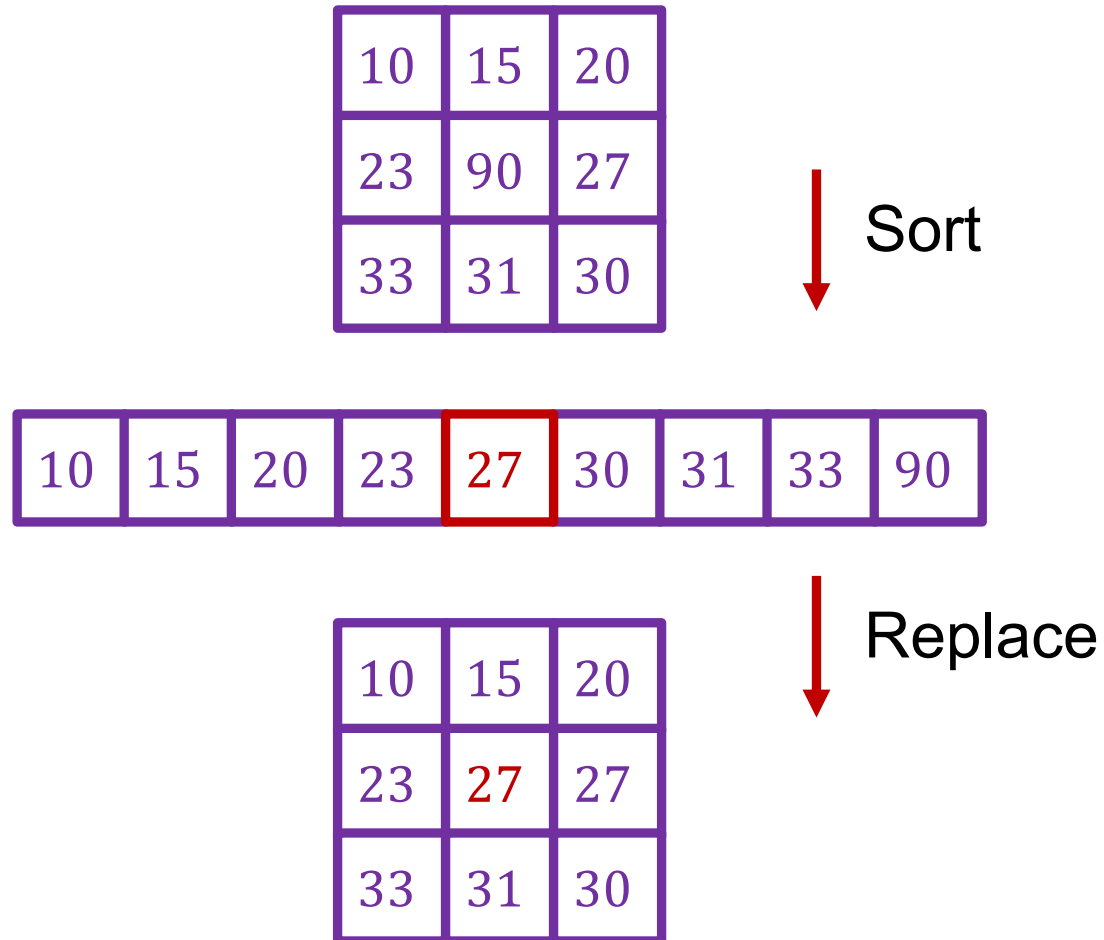
Where Gaussian filtering fails

$$\sigma = 1$$



Alternative idea: Median filtering

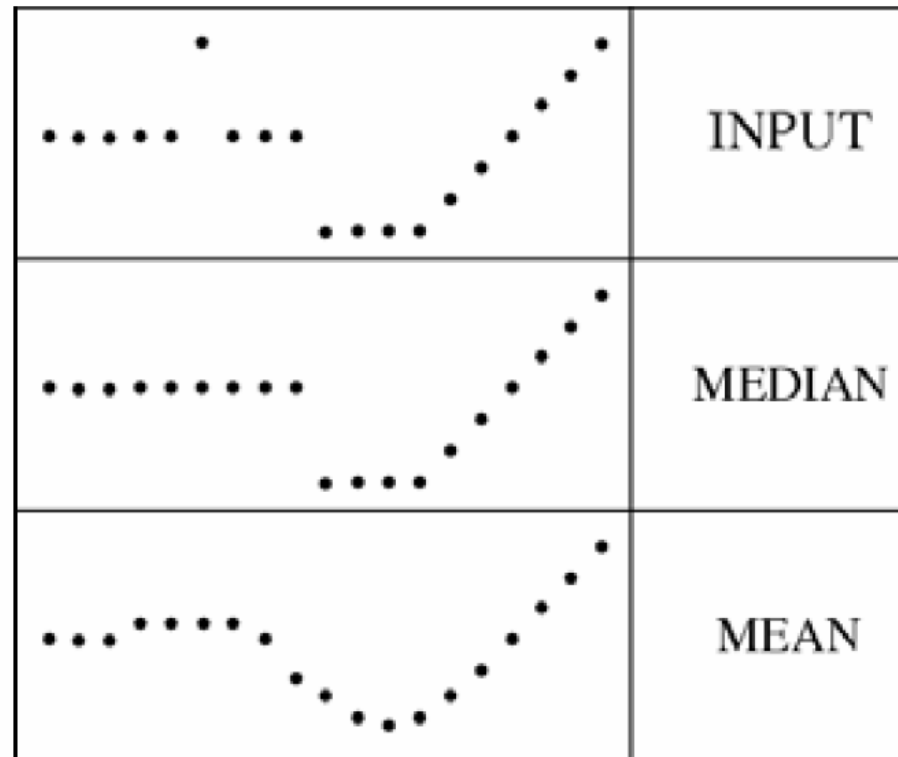
- A **median filter** operates over a window by selecting the median intensity in the window



Median filter

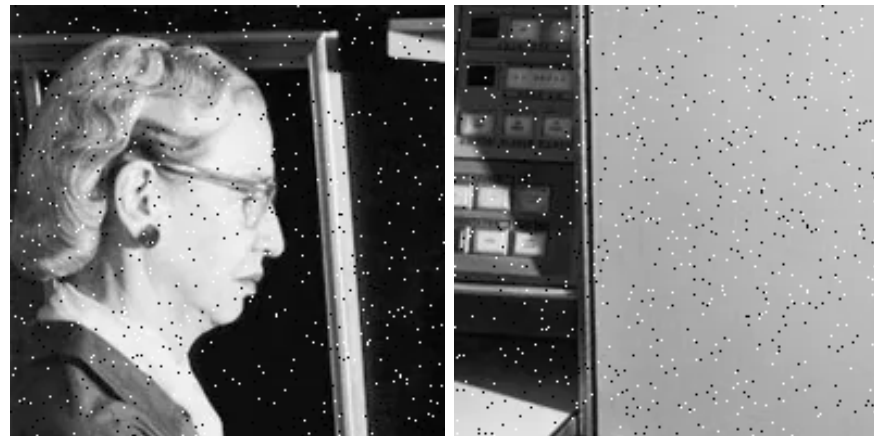
- What advantage does median filtering have over Gaussian filtering?
 - Robustness to outliers

filters have width 5 :



Applying median filter

Input image
(no filter)



Applying median filter

median
filter
(width = 3)



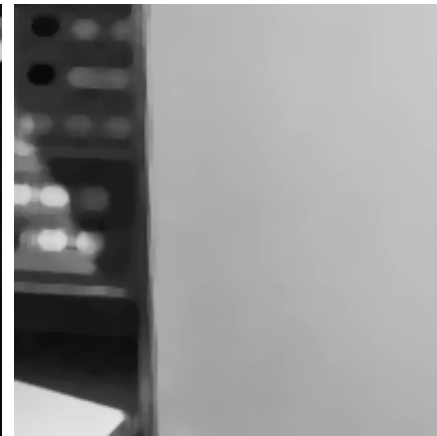
Applying median filter

median
filter
(width = 5)



Applying median filter

median
filter
(width = 7)



Is median filtering linear?

- Recall: for a linear filter,

$$\text{filter}(I_1 + I_2) = \text{filter}(I_1) + \text{filter}(I_2)$$

Is median filtering linear?

- Recall: for a linear filter,

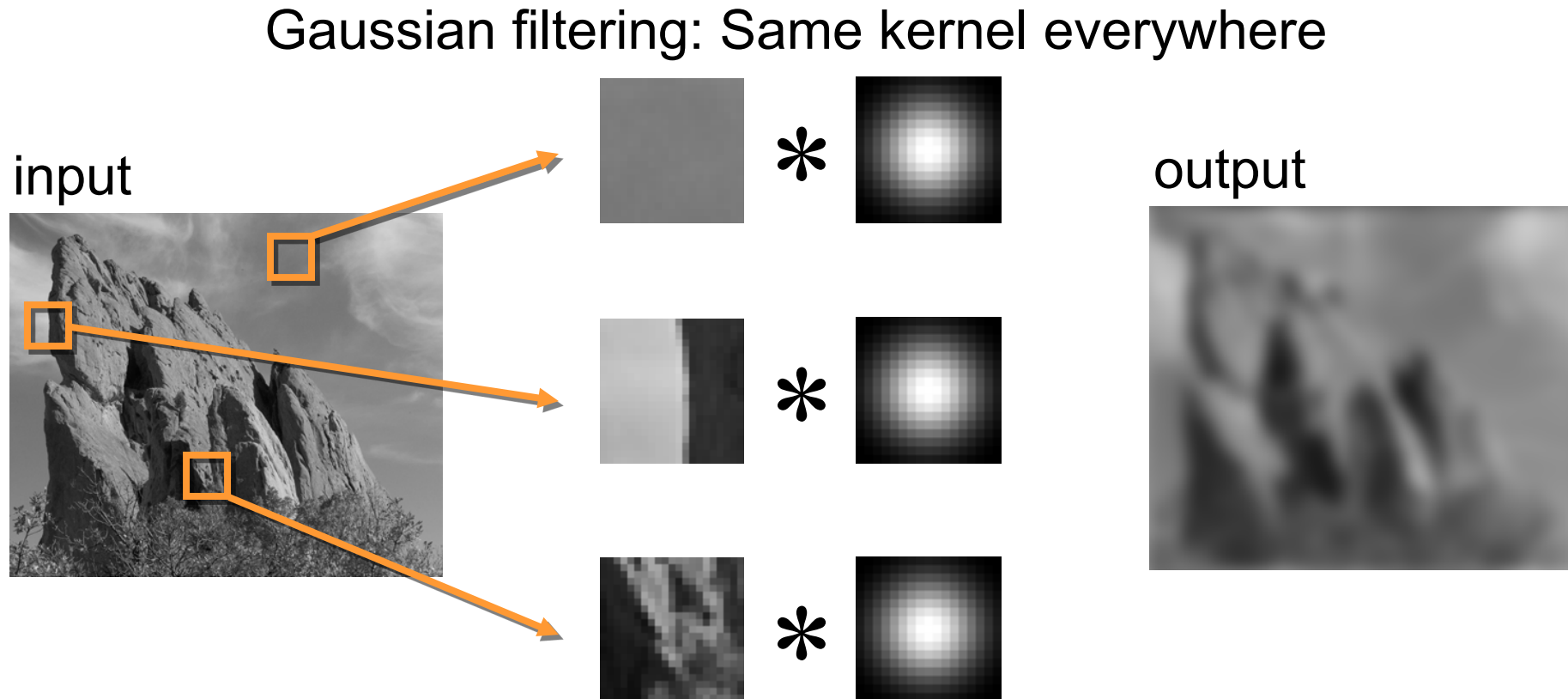
$$\text{filter}(I_1 + I_2) = \text{filter}(I_1) + \text{filter}(I_2)$$

$$\begin{array}{ccccc} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 2 & 2 & 2 \end{bmatrix} & + & \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} & = & \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix} \\ \text{median} & & & & \\ \text{filter} & & & & \\ \downarrow & & \downarrow & & \downarrow \\ 1 & + & 0 & \neq & 2 \end{array}$$

Nonlinear Filtering: Bilateral Filtering

What other kinds of filtering are there?

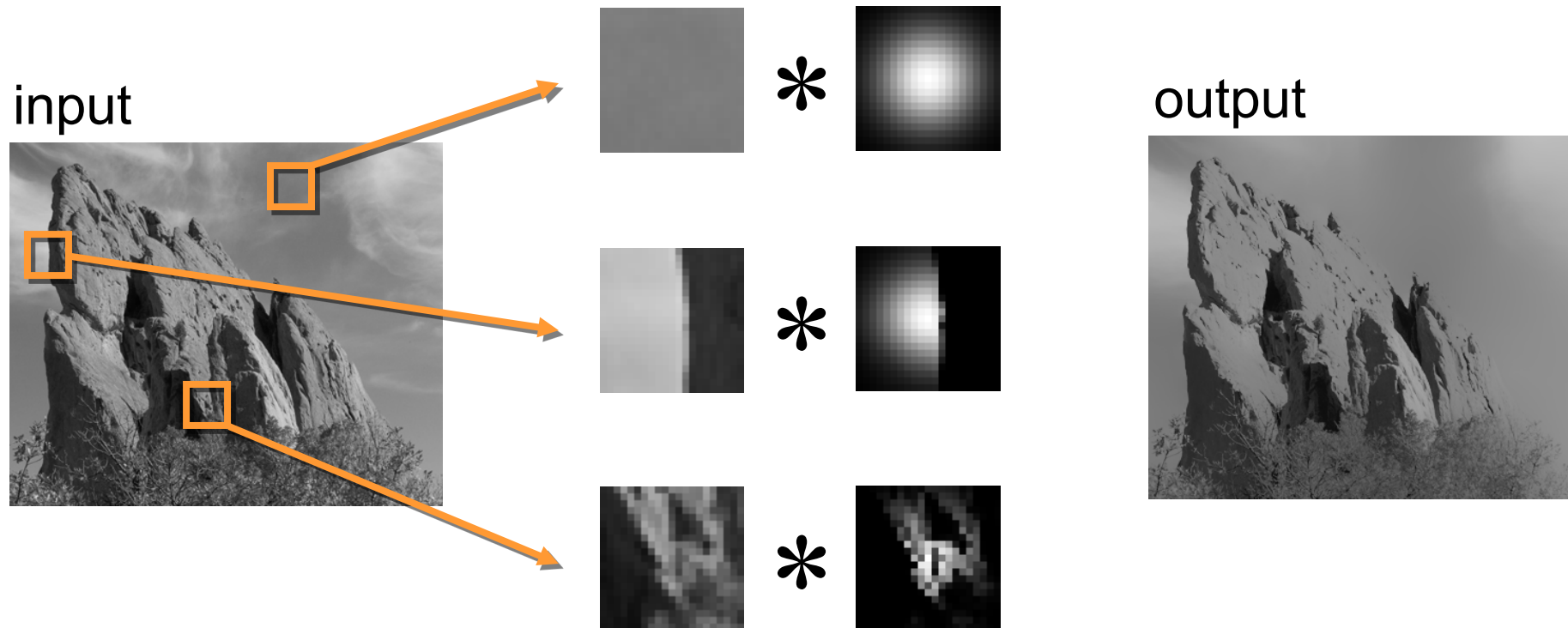
- We want something like Gaussian filtering, but with edge preservation



Bilateral filtering

[Aurich 95, Smith 97, Tomasi 98]

Kernel weights depend on the neighborhood values – no averaging across edges!



Bilateral filtering

- Compute weighted average of pixels, but the weight depends both on the distance of the filter element from the center and the difference of pixel value from the center

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

Diagram illustrating the components of the Bilateral Filtering equation:

- Normalization factor:** $\frac{1}{W_p}$ (indicated by a pink box and arrow)
- space weight:** $G_{\sigma_s}(\|p - q\|)$ (indicated by an orange box and arrow, with a corresponding 2D Gaussian plot below)
- range weight:** $G_{\sigma_r}(|I_p - I_q|)$ (indicated by a blue box and arrow, with a corresponding 1D Gaussian plot below)

Bilateral filtering

input



$\sigma_r = 0.1$



$\sigma_r = 0.25$

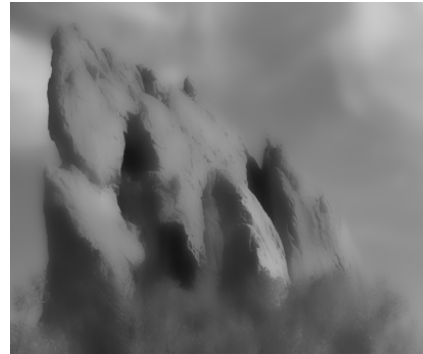


$\sigma_r = \infty$ (Gaussian blur)

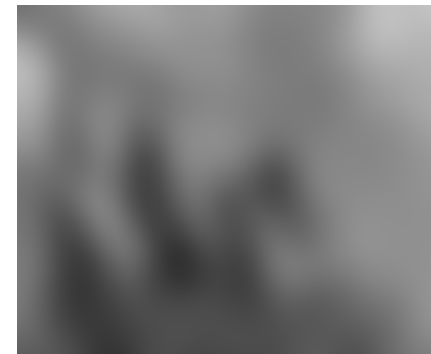
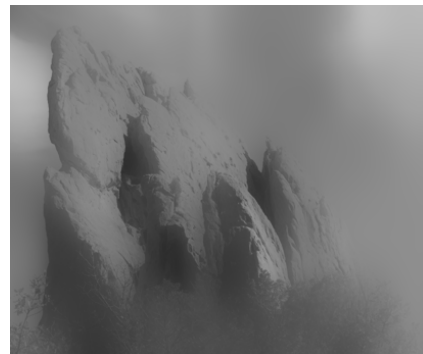


$\sigma_s = 2$

$\sigma_s = 6$



$\sigma_s = 18$



Denoising comparison

Noisy input



Median 3x3



Median 5x5



Bilateral 7x7



Application: Hybrid Images

Application: Hybrid images



A. Oliva et al. [Hybrid Images](#). SIGGRAPH 2006

Recall: Sharpening

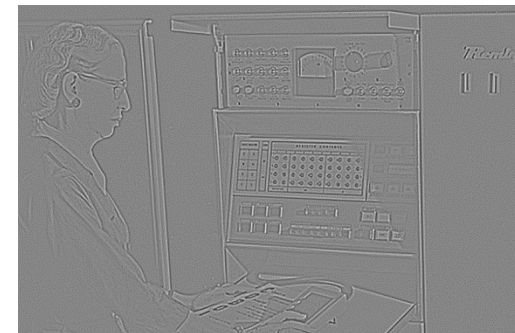
Image



Smoothed



Details



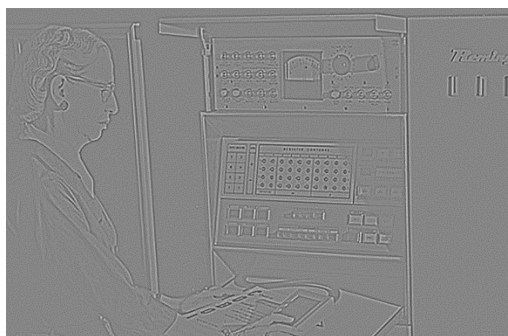
-

=

Image



Details



+ α

=

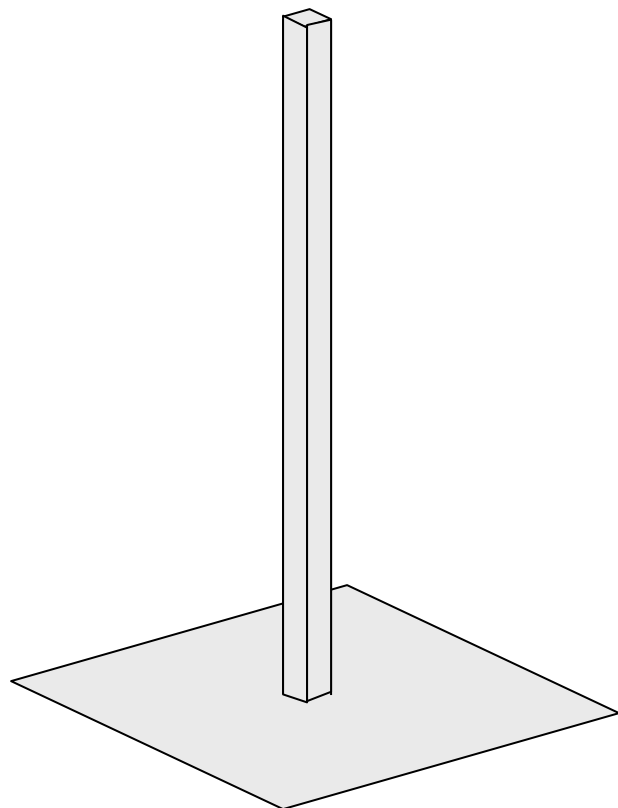
Sharpened



“Detail” filter

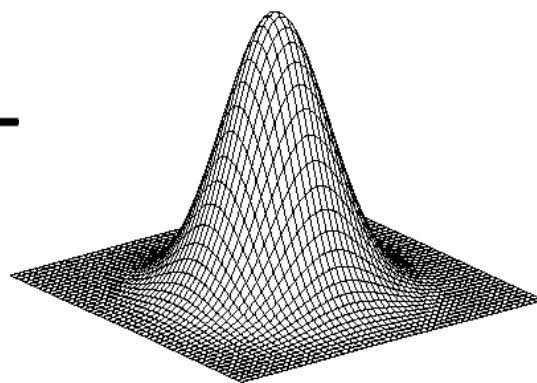
$$I - I * g = I * (e - g)$$

↑
unit impulse
(identity)



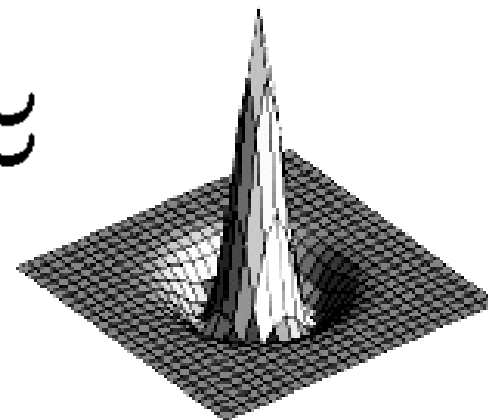
unit impulse

—



Gaussian

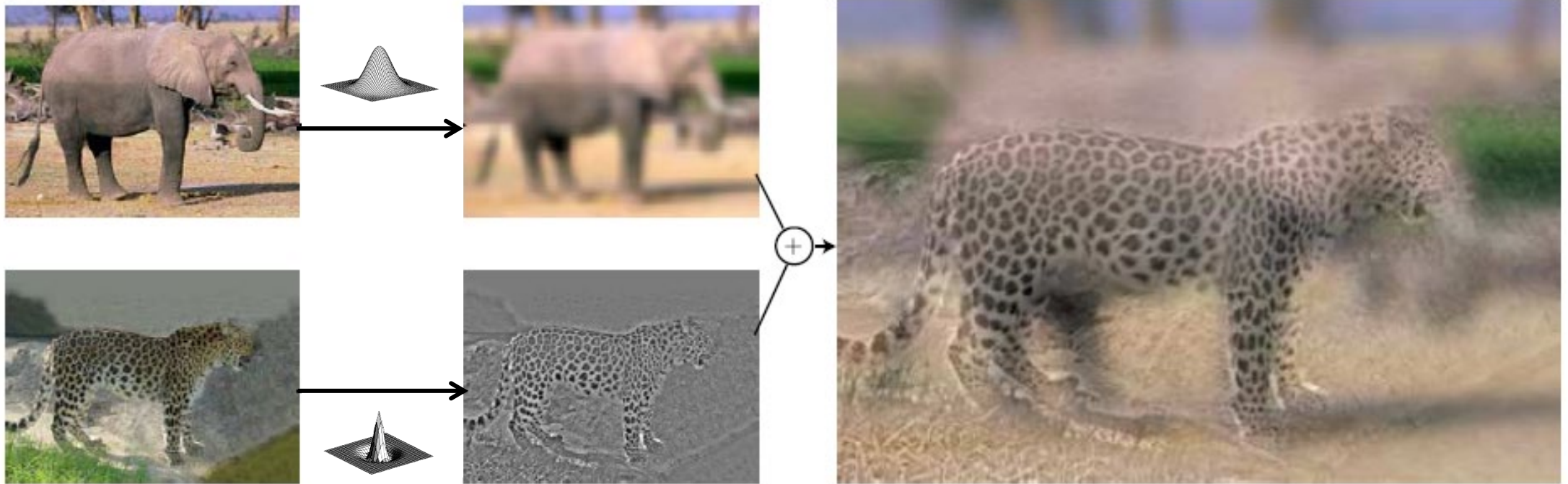
≈



Laplacian of Gaussian

Application: Hybrid images

Gaussian filter



Laplacian filter

Application: Hybrid images

Changing expression

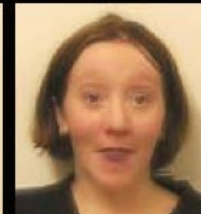


SIGGRAPH2006

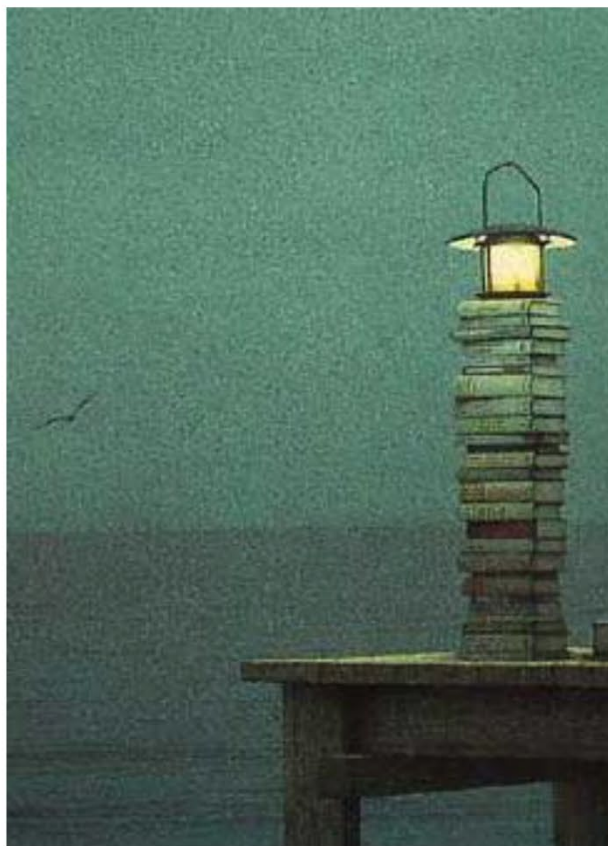
Sad



Surprised



Why it's good to read old papers



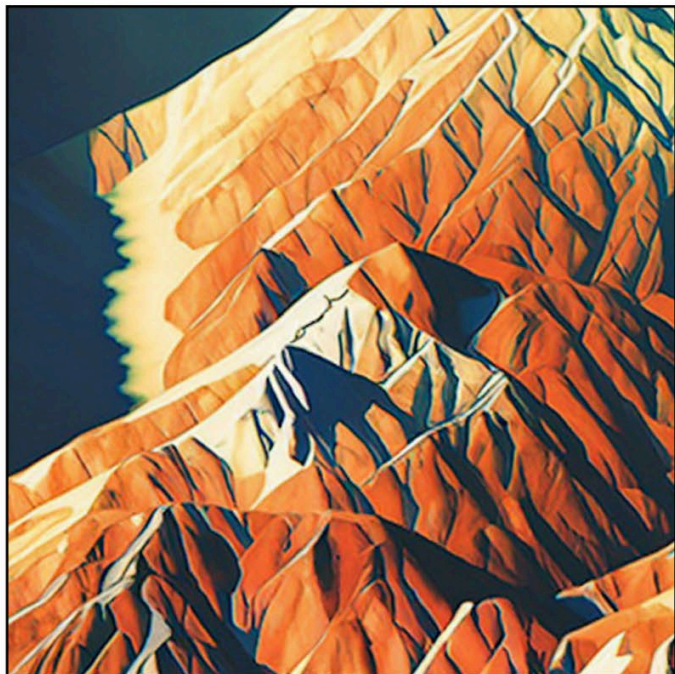
© Quint Buchholtz



Alexei (Alyosha) Efros
UC Berkeley

[RetroCV workshop at CVPR 2024](#)

Hybrid images with diffusion models



"a photo of mountains"



"a photo of a
teddy bear"



"a lithograph of
waterfalls"



"a lithograph
of a pig"



"a lithograph of
flower arrangements"



"a lithograph
of a panda"

Hybrid images with diffusion models



"a photo of
houseplants"



"a photo of a rabbit"



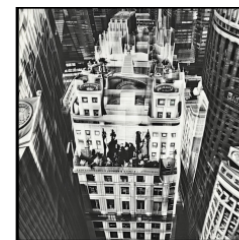
"a photo of
new york city"



"a photo of
marilyn
monroe"



"a photo of
an old man"



"a photo of
john lennon"

Hybrid images with diffusion models

Triple hybrids

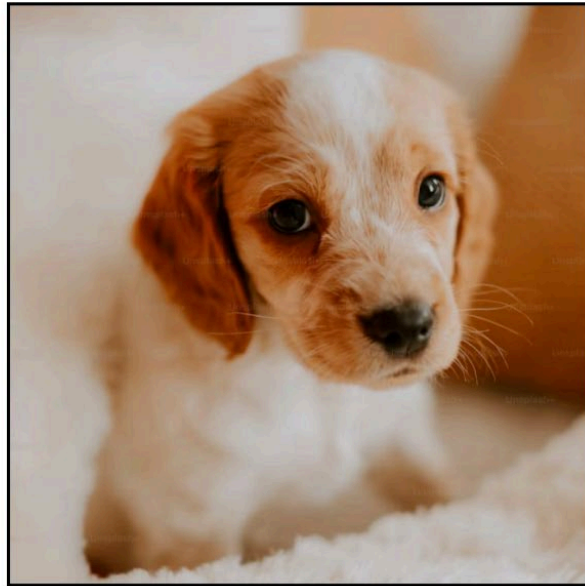
"a photo of
waterfalls"



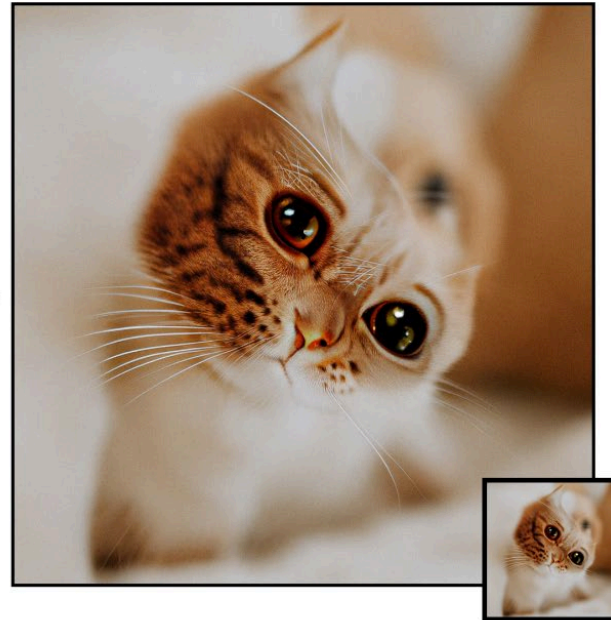
Hybrid images with diffusion models

Inverse hybrids

Low Pass:
[Real Image]



High Pass:
"a photo of a cat"



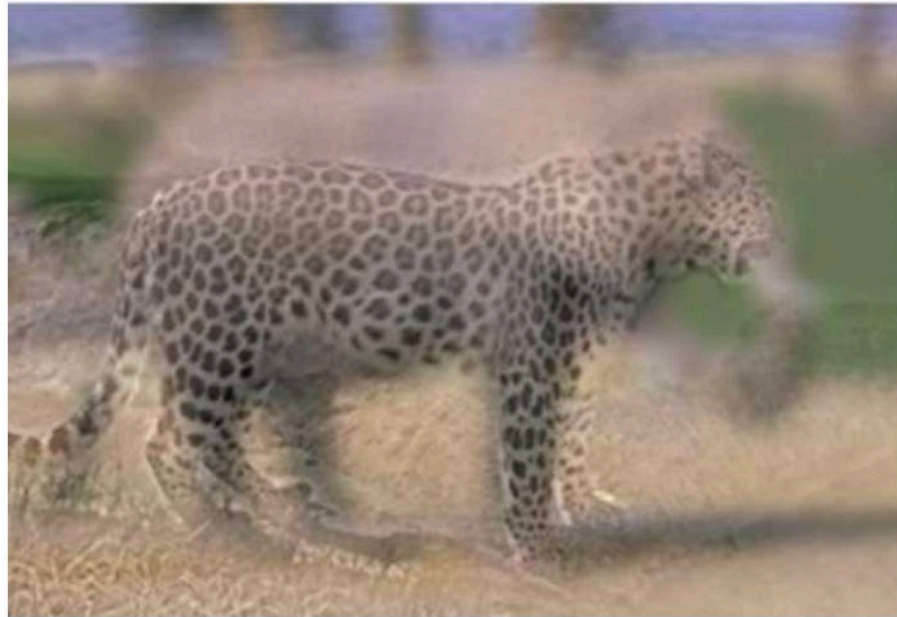
Hybrid images with diffusion models

Comparison with Oliva et al. (2006)

“leopard” / “elephant”



Geng et al. (2024)



Oliva et al. (2006)

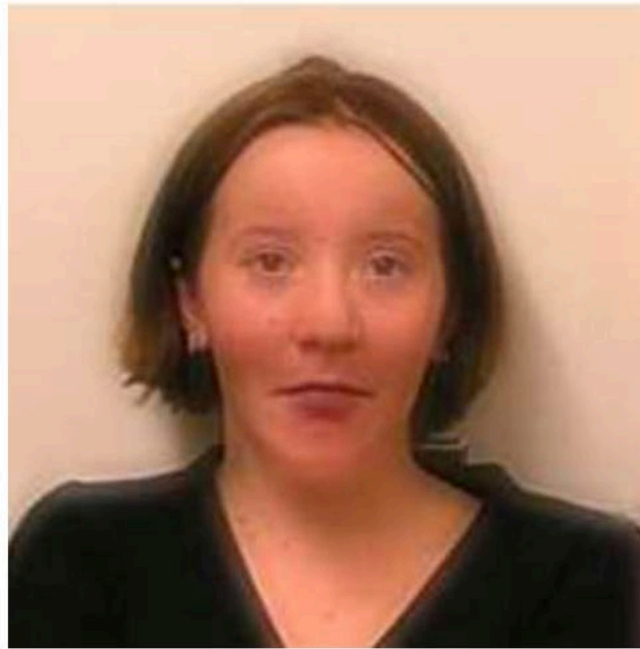
Hybrid images with diffusion models

Comparison with Oliva et al. (2006)

“sad woman” / “happy woman”



Geng et al. (2024)

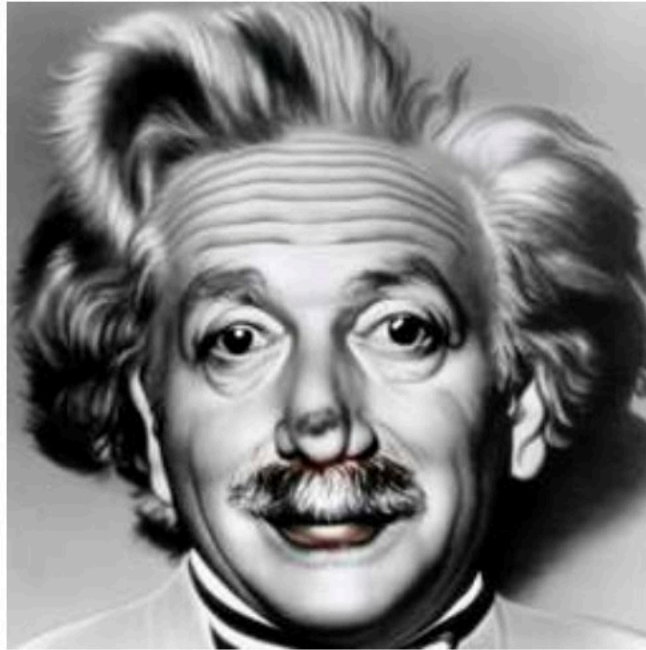


Oliva et al. (2006)

Hybrid images with diffusion models

Comparison with Oliva et al. (2006)

“Albert Einstein” / “Marilyn Monroe”



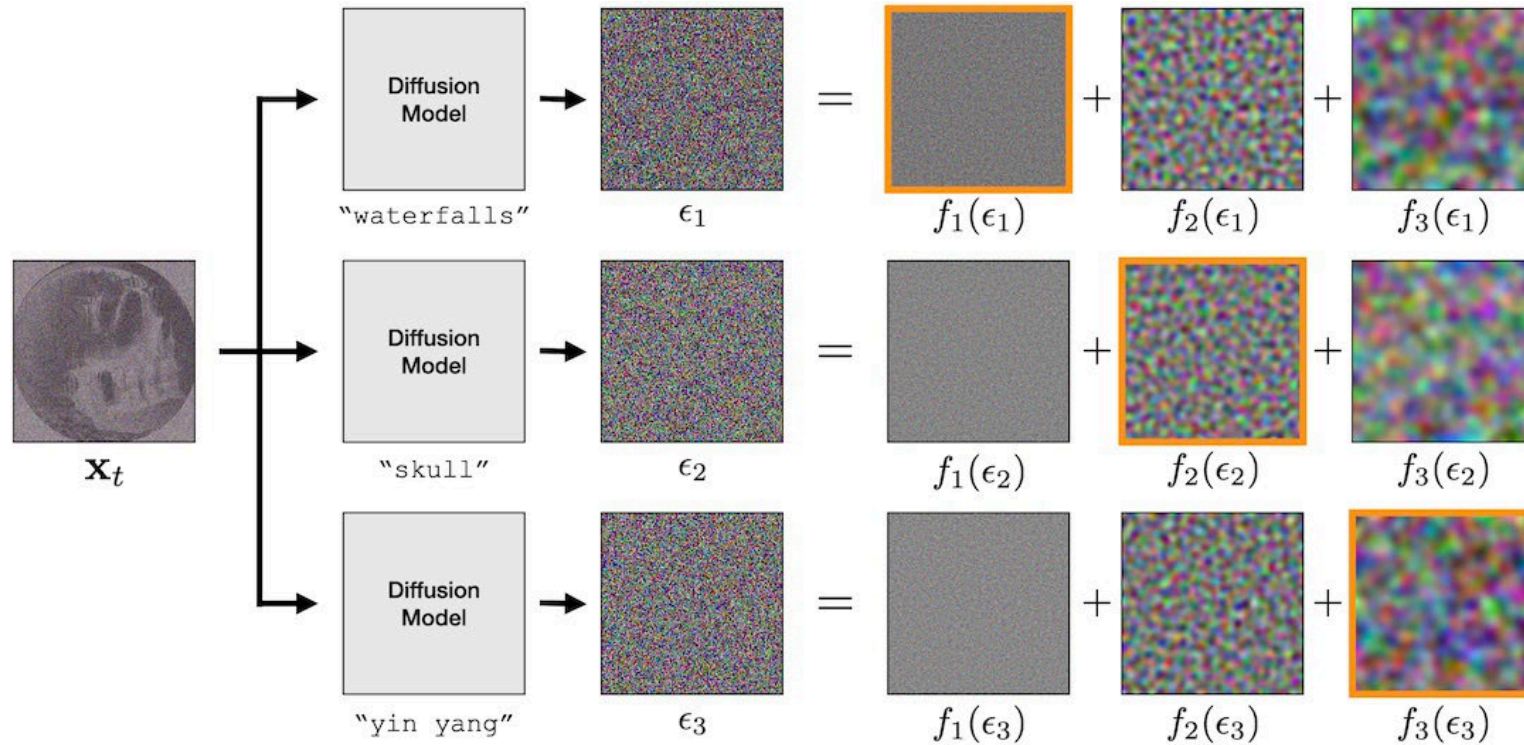
Geng et al. (2024)



Oliva et al. (2006)

Hybrid images with diffusion models

Factorized diffusion



$$\mathbf{x} = \sum_i f_i(\mathbf{x})$$

$$\tilde{\epsilon} = f_1(\epsilon_1) + f_2(\epsilon_2) + f_3(\epsilon_3)$$

Questions?

The only stupid question is the one you were afraid to ask but never did.

-Rich Sutton

Thank You